



République Tunisienne  
Ministère de l'Enseignement Supérieur  
et de la Recherche Scientifique  
École Supérieure Privée d'ingénierie et de technologie  
TEK-UP



## RAPPORT DE PROJET DE FIN D'ÉTUDES

Présenté en vue de l'obtention du  
Diplôme National d'Ingénieur en Sciences Appliquées et Technologiques  
Spécialité : Filière

Réalisé par

XXXXXXXXXXXXXXXXXX

---

### Conception et Développement d'une Plateforme Centralisée de Gestion et de Pilotage Financier des Projets Informatiques (PMS)

---

Encadrant professionnel : **Monsieur Prénom NOM**

Ingénieur Informatique

Encadrant académique : **Monsieur ZZZZZZZ**

Poste



J'autorise l'étudiant à faire le dépôt de son rapport de stage en vue d'une soutenance.

Encadrant professionnel, **M. XXXXXXXX**

**Signature et cachet**

J'autorise l'étudiant à faire le dépôt de son rapport de stage en vue d'une soutenance.

Encadrant académique, **M. YYYYYYY**

**Signature**

# Dédicace

# Remerciements

# Table des matières

|                                                    |           |
|----------------------------------------------------|-----------|
| <b>General Introduction</b>                        | <b>1</b>  |
| <b>1 Project Framework</b>                         | <b>3</b>  |
| 1.1 Host organization . . . . .                    | 4         |
| 1.1.1 Presentation of ST2i . . . . .               | 4         |
| 1.1.2 Areas of expertise . . . . .                 | 4         |
| 1.1.3 Services offered . . . . .                   | 4         |
| 1.1.4 Organizational structure . . . . .           | 5         |
| 1.2 Project context . . . . .                      | 5         |
| 1.2.1 The assigned subject . . . . .               | 5         |
| 1.2.2 Project objectives . . . . .                 | 6         |
| 1.3 Study of the existing system . . . . .         | 6         |
| 1.3.1 The current process . . . . .                | 6         |
| 1.3.2 Analysis of the reference workbook . . . . . | 7         |
| 1.3.3 Existing market solutions . . . . .          | 11        |
| 1.3.4 Comparative synthesis . . . . .              | 12        |
| 1.3.5 Critique of the existing system . . . . .    | 13        |
| 1.4 Problem statement . . . . .                    | 13        |
| 1.5 Proposed solution . . . . .                    | 14        |
| 1.6 Project management methodology . . . . .       | 14        |
| 1.6.1 The Scrum framework . . . . .                | 14        |
| 1.6.2 Scrum roles . . . . .                        | 14        |
| 1.6.3 Scrum ceremonies . . . . .                   | 15        |
| 1.6.4 Adaptation to this project . . . . .         | 15        |
| <b>2 Requirements Analysis and Specification</b>   | <b>17</b> |
| 2.1 Actor Identification . . . . .                 | 18        |
| 2.2 Functional Requirements . . . . .              | 18        |
| 2.3 Non-Functional Requirements . . . . .          | 19        |
| 2.4 Business Rules . . . . .                       | 20        |
| 2.5 The Dynamic Authorization Model . . . . .      | 21        |
| 2.6 Global Use Case Diagram . . . . .              | 21        |

---

|          |                                                                  |           |
|----------|------------------------------------------------------------------|-----------|
| 2.7      | Product Backlog . . . . .                                        | 22        |
| 2.8      | Release and Sprint Planning . . . . .                            | 25        |
| 2.8.1    | Schedule . . . . .                                               | 26        |
| <b>3</b> | <b>System Architecture and Design</b>                            | <b>27</b> |
| 3.1      | Architectural Style . . . . .                                    | 28        |
| 3.1.1    | A Three-Tier Modular Monolith . . . . .                          | 28        |
| 3.1.2    | Layered Organisation . . . . .                                   | 29        |
| 3.2      | Cross-Cutting Design Decisions . . . . .                         | 30        |
| 3.2.1    | Authentication and Session Revocation . . . . .                  | 30        |
| 3.2.2    | Dynamic Authorization . . . . .                                  | 30        |
| 3.2.3    | A Hybrid Indicator Engine . . . . .                              | 30        |
| 3.2.4    | Multi-Currency Handling . . . . .                                | 30        |
| 3.2.5    | Auditability and Logical Deletion . . . . .                      | 31        |
| 3.3      | Domain Model . . . . .                                           | 31        |
| 3.4      | Database Design . . . . .                                        | 32        |
| 3.4.1    | Modelling Conventions . . . . .                                  | 32        |
| 3.4.2    | Logical Model . . . . .                                          | 32        |
| 3.4.3    | Monthly Time Series and Indicator Snapshots . . . . .            | 33        |
| 3.4.4    | Reference Data and Migration Strategy . . . . .                  | 33        |
| 3.5      | Deployment Architecture . . . . .                                | 34        |
| 3.6      | Technical Environment . . . . .                                  | 34        |
| <b>4</b> | <b>Authentication and Access Control</b>                         | <b>36</b> |
| 4.1      | Release 1 backlog . . . . .                                      | 37        |
| 4.2      | Sprint 1 — Authentication and technical foundation . . . . .     | 38        |
| 4.2.1    | Sprint 1 use case diagram . . . . .                              | 38        |
| 4.2.2    | Sprint 1 analysis and design . . . . .                           | 38        |
| 4.2.3    | Sprint 1 realization . . . . .                                   | 39        |
| 4.3      | Sprint 2 — Dynamic access control, users and resources . . . . . | 40        |
| 4.3.1    | Sprint 2 use case diagram . . . . .                              | 40        |
| 4.3.2    | Sprint 2 analysis and design . . . . .                           | 41        |
| 4.3.3    | Sprint 2 realization . . . . .                                   | 41        |

---

---

|          |                                                                    |           |
|----------|--------------------------------------------------------------------|-----------|
| <b>5</b> | <b>Project and Operational Management</b>                          | <b>43</b> |
| 5.1      | Release 2 backlog . . . . .                                        | 44        |
| 5.2      | Sprint 3 — Project management and lifecycle . . . . .              | 45        |
| 5.2.1    | Sprint 3 use case diagram . . . . .                                | 45        |
| 5.2.2    | Sprint 3 analysis and design . . . . .                             | 46        |
| 5.2.3    | Sprint 3 realization . . . . .                                     | 48        |
| 5.3      | Sprint 4 — Team assignment and workload . . . . .                  | 48        |
| 5.3.1    | Sprint 4 use case diagram . . . . .                                | 48        |
| 5.3.2    | Sprint 4 analysis and design . . . . .                             | 49        |
| 5.3.3    | Sprint 4 realization . . . . .                                     | 50        |
| 5.4      | Sprint 5 — Agile planning . . . . .                                | 50        |
| 5.4.1    | Sprint 5 use case diagram . . . . .                                | 50        |
| 5.4.2    | Sprint 5 analysis and design . . . . .                             | 50        |
| 5.4.3    | Sprint 5 realization . . . . .                                     | 52        |
| <b>6</b> | <b>Financial Management and Governance</b>                         | <b>53</b> |
| 6.1      | Release 3 backlog . . . . .                                        | 54        |
| 6.2      | Sprint 6 — Billing and missions . . . . .                          | 55        |
| 6.2.1    | Sprint 6 use case diagram . . . . .                                | 55        |
| 6.2.2    | Sprint 6 analysis and design . . . . .                             | 55        |
| 6.2.3    | Sprint 6 realization . . . . .                                     | 56        |
| 6.3      | Sprint 7 — Project governance . . . . .                            | 57        |
| 6.3.1    | Sprint 7 use case diagram . . . . .                                | 57        |
| 6.3.2    | Sprint 7 analysis and design . . . . .                             | 57        |
| 6.3.3    | Sprint 7 realization . . . . .                                     | 58        |
| 6.4      | Sprint 8 — Internal quote and indicator engine . . . . .           | 58        |
| 6.4.1    | Sprint 8 use case diagram . . . . .                                | 58        |
| 6.4.2    | Sprint 8 analysis and design . . . . .                             | 58        |
| 6.4.3    | Sprint 8 realization . . . . .                                     | 60        |
| <b>7</b> | <b>Consolidation, Delivery and Deployment</b>                      | <b>61</b> |
| 7.1      | Release 4 backlog . . . . .                                        | 62        |
| 7.2      | Sprint 9 — Interface consolidation, testing and security . . . . . | 63        |
| 7.2.1    | Sprint 9 use case diagram . . . . .                                | 63        |
| 7.2.2    | Sprint 9 analysis and design . . . . .                             | 63        |

---

|                                            |                                                                |           |
|--------------------------------------------|----------------------------------------------------------------|-----------|
| 7.2.3                                      | Sprint 9 realization . . . . .                                 | 64        |
| 7.3                                        | Sprint 10 — Containerization and continuous delivery . . . . . | 65        |
| 7.3.1                                      | Sprint 10 use case diagram . . . . .                           | 65        |
| 7.3.2                                      | Sprint 10 analysis and design . . . . .                        | 65        |
| 7.3.3                                      | Sprint 10 realization . . . . .                                | 66        |
| 7.4                                        | Final state of the product . . . . .                           | 67        |
| <b>General Conclusion and Perspectives</b> |                                                                | <b>69</b> |
| <b>Webographie</b>                         |                                                                | <b>71</b> |

# Table des figures

|     |                                                                       |    |
|-----|-----------------------------------------------------------------------|----|
| 2.1 | Global use case diagram of the PMS platform . . . . .                 | 22 |
| 3.1 | Functional modules and their main dependencies . . . . .              | 29 |
| 3.2 | Domain model — principal classes and their associations . . . . .     | 31 |
| 3.3 | Class diagram — security and access control . . . . .                 | 32 |
| 3.4 | Deployment architecture . . . . .                                     | 34 |
| 5.1 | Class diagram — projects and teams . . . . .                          | 46 |
| 5.2 | State diagram — project lifecycle . . . . .                           | 46 |
| 5.3 | Sequence diagram of the “Create a project” use case . . . . .         | 47 |
| 5.4 | Sequence diagram of the “Assign a developer” use case . . . . .       | 49 |
| 5.5 | Class diagram — agile planning . . . . .                              | 51 |
| 6.1 | Class diagram — financial management . . . . .                        | 56 |
| 6.2 | Class diagram — project governance . . . . .                          | 57 |
| 6.3 | Class diagram — workload and indicator engine . . . . .               | 58 |
| 6.4 | Sequence diagram — workload entry and indicator computation . . . . . | 59 |

# Liste des tableaux

|     |                                                                                       |    |
|-----|---------------------------------------------------------------------------------------|----|
| 1.1 | Functional structure of the Excel project review workbook . . . . .                   | 7  |
| 1.2 | Traceability from the Excel workbook to the PMS modules . . . . .                     | 11 |
| 1.3 | Comparison of existing tools against the requirements of the reference workbook . . . | 12 |
| 2.1 | PMS actors and their boundaries . . . . .                                             | 18 |
| 2.2 | Functional requirements by module . . . . .                                           | 19 |
| 2.3 | Summary of business rules . . . . .                                                   | 21 |
| 2.4 | Product backlog of the PMS platform . . . . .                                         | 25 |
| 2.5 | Distribution of the backlog across releases and sprints . . . . .                     | 25 |
| 2.6 | Schedule of the preparation phase and the ten sprints . . . . .                       | 26 |
| 3.1 | Catalogue of entities by domain . . . . .                                             | 33 |
| 3.2 | Technical environment of the project . . . . .                                        | 34 |
| 4.1 | Backlog of release 1 . . . . .                                                        | 38 |
| 5.1 | Backlog of release 2 . . . . .                                                        | 45 |
| 6.1 | Backlog of release 3 . . . . .                                                        | 55 |
| 7.1 | Backlog of release 4 . . . . .                                                        | 62 |
| 7.2 | Security review by vulnerability category . . . . .                                   | 65 |
| 7.3 | From the reference workbook to the delivered platform . . . . .                       | 68 |

# List of Abbreviations

- **API** = Application Programming Interface
- **BR** = Business Rule
- **BRS** = Business Requirements Specification
- **EAC** = Estimate At Completion
- **ETC** = Estimate To Complete
- **EV** = Earned Value
- **FAE** = Facture À Établir — revenue earned but not yet invoiced
- **FCFA** = Franc of the African Financial Community
- **FP** = Fixed Price — contract billed at an agreed total
- **FR** = Functional Requirement
- **HTTP** = Hypertext Transfer Protocol
- **JPA** = Java Persistence API
- **JWT** = JSON Web Token
- **KPI** = Key Performance Indicator
- **NFR** = Non-Functional Requirement
- **ORM** = Object-Relational Mapping
- **PFE** = Projet de Fin d'Études — final-year graduation project
- **PMS** = Project Management System
- **PPR** = Provision Pour Risques — risk provision withheld from margin
- **RBAC** = Role-Based Access Control
- **REST** = Representational State Transfer
- **SQL** = Structured Query Language
- **SRS** = Software Requirements Specification

- **TCC** = Taux de Coût Chargé — fully loaded cost rate of a resource
- **TND** = Tunisian Dinar
- **UC** = Use Case
- **UML** = Unified Modeling Language
- **US** = User Story

# General Introduction

In a digital services company, profitability is decided project by project. Each engagement sells a volume of effort at a negotiated price, consumes resources whose cost is known, and is invoiced according to a contractual schedule. Steering therefore means keeping three quantities under control at once : the workload sold, the effort actually consumed, and the revenue invoiced. Losing sight of any one of them means discovering a loss-making project only once it is finished.

The host company steers this equilibrium through an Excel workbook, a “project review sheet” of some twenty interconnected tabs and more than two thousand formulas, duplicated for every project. This workbook is remarkable in what it encodes : loaded cost rates per resource and per year, monthly workload plans, billing milestones as a percentage of the contract, mission expenses, governance registers and a complete earned-value indicator engine. It is, in practice, the company’s real specification.

It is also a fragile medium. One file per project means no single source of truth ; formulas spanning tabs break silently, and the file provided for this project indeed contains cells in error ; consolidating the portfolio is impossible without reopening every workbook ; and a spreadsheet cannot prevent a developer from reading the margin of the project they work on.

This final year project addresses that gap through **PMS** (*Project Management System*), a centralised web platform that automates the business logic carried by the workbook. Two requirements shaped it. The first is fidelity : the platform must reproduce the financial model faithfully (cost rates, workload plan, estimate at completion, margins, billing progress) rather than a simplified approximation of it. The second is configurability : the authorization policy must be modifiable as data, without redeploying the application, so that the platform can absorb the evolution of the company’s rules.

The platform was built incrementally, in ten successive sprints grouped into four releases, each delivering a demonstrable increment of the product. This report follows that construction.

**Chapter 1** situates the project : the host organization, the analysis of the existing Excel-based process, including the representative sheets from which each module derives, the resulting problem statement, and the methodology adopted.

**Chapter 2** specifies the system : actors and their boundaries, functional and non-functional requirements, business rules, the dynamic authorization model, the global use case diagram, and the product backlog and its distribution across the development sprints.

**Chapter 3** presents the technical design : the architectural style and its justification, the cross-cutting design decisions, the domain model and the relational schema.

**Chapters 4 to 7** present the realisation, release by release : the foundation and security of Release 1, the operational management of Release 2, the financial and governance value of Release 3, and the consolidation, testing and deployment of Release 4.

A **general conclusion** assesses the result against the initial objectives and outlines the perspectives opened by the platform.

---

# PROJECT FRAMEWORK

---

## Plan

|   |                                          |    |
|---|------------------------------------------|----|
| 1 | Host organization . . . . .              | 4  |
| 2 | Project context . . . . .                | 5  |
| 3 | Study of the existing system . . . . .   | 6  |
| 4 | Problem statement . . . . .              | 13 |
| 5 | Proposed solution . . . . .              | 14 |
| 6 | Project management methodology . . . . . | 14 |

## Introduction

This first chapter situates the project in its context. It presents the host organization and the subject assigned by that organization, analyses the project-tracking process currently in place, which rests on Excel workbooks, and compares it with the tools available on the market. From this study follow the problem statement, the proposed solution, and the management methodology adopted to build it.

### 1.1 Host organization

#### 1.1.1 Presentation of ST2i

This work was carried out at **ST2i**, a consulting firm created in the early 1990s and a member of **STUDI Group**, an international multidisciplinary engineering group. ST2i positions itself on high value-added segments in information technology, geomatics and the new information and communication technologies.

The company has carried out projects in more than fifteen countries : Tunisia, Algeria, Mali, Côte d'Ivoire, Senegal, Burkina Faso, Benin, Togo, Cameroon, Guinea, the Democratic Republic of the Congo, Mauritania, Saudi Arabia and Canada, among others. Its head office is in Tunis, in the Lac II business district.

#### 1.1.2 Areas of expertise

ST2i operates in four complementary areas.

- **Information systems.** Auditing existing systems and designing strategic and operational master plans, particularly for public administrations and public establishments.
- **Geomatics and geographic information systems.** A multidisciplinary team of geomatics experts, developers and technicians covering a broad range of specialised software solutions.
- **Telecommunications and ICT.** Auditing existing networks, steering projects under way, and defining the specifications of new solutions and networks.
- **Engineering software.** Standard, open products that respect the regulations in force and the working methods of the various professional disciplines.

#### 1.1.3 Services offered

The firm structures its offer around three service lines.

- **Studies and consulting** : strategic and sectoral studies, information system urbanisation, networks and digital development of territories.
- **Project-owner assistance** : support in the controlled conduct of complex projects, covering the validation of execution projects, coordination, monitoring and control, technical acceptance, dispute resolution and project closure.
- **Development and integration of solutions** : design, implementation and deployment of computing and telecommunications solutions across a range of environments and architectures.

The third line is the one that concerns this project directly, and it is also the activity whose economics the platform described in this report is meant to steer.

#### 1.1.4 Organizational structure

The firm is organised around a management team and multidisciplinary technical teams that bring together geomatics experts, developers and technicians. This combination of profiles within a single team is what allows ST2i to carry a project from the initial study through to the delivered solution, and it is also what makes the steering of workload across several simultaneous projects a concrete daily problem rather than an abstract one.

## 1.2 Project context

### 1.2.1 The assigned subject

This work constitutes a Final Year Project submitted for the National Engineering Diploma. It responds to subject 6 of the ST2i PFE Book 2025–2026, on the *C.DEV* track : *design and development of a centralised platform for the management and steering of information technology projects*.

As stated by the company, the platform must centralise all information attached to IT projects, that is owners, project managers, teams, budgets, specifications, workload in man-days and expenses. It must offer complete financial monitoring covering costs, forecasts, variances and gains or losses ; operational monitoring covering progress, delays, milestones and payments ; and agile planning through sprints and a backlog. It must finally provide dynamic dashboards showing the state of the project portfolio, team performance and risks. The stated aim is to optimise steering, automate the collection of information and improve decision-making.

The technical skills listed for the subject are databases, Java, Spring Boot and Angular, which correspond exactly to the stack retained in Chapter 3. The project ran from 5 January to 30 August 2026.

This framing matters for a reason specific to the subject : the company that steers IT projects for its clients is itself the user of the platform. The requirements were therefore not gathered from hypothetical users but read directly from the instrument the company already uses every day, which is the object of the study below.

### 1.2.2 Project objectives

Four objectives were set at the outset.

- Replace the dispersion of one workbook per project by a single, shared and reliable source of information.
- Reproduce the financial model of the existing process faithfully rather than approximate it, since that model is the company's real specification.
- Partition access to information, so that the financial data of a project is visible only to those entitled to it, which a spreadsheet cannot enforce.
- Keep the authorization policy configurable, so that a change in the company's rules does not require a new version of the software.

## 1.3 Study of the existing system

### 1.3.1 The current process

Each project is tracked through an Excel workbook, the “project review sheet”, comprising twenty-one interconnected tabs. For a given project, this workbook centralises all steering information : project identification and budget, resource cost rates, planned workload, actual workload, billing milestones, missions, risks, deliverables and dashboards.

| Tab family                                      | Business role                                                                                                |
|-------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| Identification                                  | Project identification sheet : client, contract, budget, currency, duration, project manager, workload sold. |
| Costs (TCC, Forecast Cost, Updated Actual Cost) | Computation of the daily cost of resources and of forecast and actual costs per month.                       |
| Billing (Progress, Summary)                     | Billing milestones as a percentage of the contract, tracking of invoiced and settled amounts.                |

| Tab family                                                          | Business role                                                  |
|---------------------------------------------------------------------|----------------------------------------------------------------|
| Missions                                                            | Computation of travel costs (per diem, ticket, accommodation). |
| Steering (Statistics, Dashboard, Glossary)                          | Consolidation of indicators and KPI definitions.               |
| Governance (Risks, Deliverables, Stakeholders, Changes, Amendments) | Registers tracking the life of the project.                    |

**TABLEAU 1.1** : Functional structure of the Excel project review workbook

### 1.3.2 Analysis of the reference workbook

The workbook provided by the company (reference **F-AFF-13**, version 03) is the true functional specification of the platform : it contains twenty-one sheets and more than two thousand formulas that encode the company's real steering logic. Its analysis constituted the preparation phase of this project, and each PMS module derives directly from one or more of these sheets.

The eight most representative sheets are presented below. The figures shown are **anonymised** : client name, contract reference, staff names and the amounts of the internal quote have been replaced by neutral values, as this information is covered by the confidentiality of the host company and of its client. The structure, the formulas and the business logic, the only elements relevant to this analysis, are preserved.

#### 1.3.2.1 Project identification sheet

**What it represents.** The identity card of the project : client, contract reference, funder, business model (alone or in consortium), engagement type (fixed price or time & material), project manager, start and end dates, contract duration, project currency and initial budget.

**Business purpose.** It is the entry point of every project review. It fixes the contractual and financial baseline against which all later indicators are measured.

**Key information and calculations.** The sheet converts the budget from the project currency (here FCFA) into the reporting currency (TND) using a contractual exchange rate ; it computes the risk provision  $PPR = \text{Budget} \times 5\%$ , the penalty provision (PPP), the total cost of the internal quote and the *sold net margin* ( $\text{Selling price} - \text{Total cost} / \text{Selling price}$ ). It also carries the amendment fields, which adjust the total budget.

**Derived PMS functionality.** The *Project management* module : project creation with code, client, funder, engagement type, multi-currency budget, provisions and sold workload ; and the *Internal*

*quote*, which reproduces this economic baseline.

### 1.3.2.2 Cost rate sheet (TCC)

**What it represents.** The daily loaded cost (*Taux de Coût Chargé*) of each resource, recorded per year.

**Business purpose.** It converts a human resource into a unit cost. Every cost computation in the workbook, forecast and actual alike, ultimately multiplies man-days by a value taken from this sheet.

**Key information and calculations.** The daily rate is derived from the annual loaded cost by  $TCC_{jour} = (\text{annual loaded cost} \times 1.7) / \text{working days per year}$ , where the overhead coefficient covers employer charges and the number of working days differs from one year to the next (22 in 2024, 20 in 2025 in the reference file). A validated historical value must remain immutable, otherwise past reviews would change retroactively.

**Derived PMS functionality.** The *Resource and yearly cost rate* module (Sprint 2) : one rate per resource and per year, with the overhead coefficient and the number of working days externalised as configurable business parameters rather than hard-coded constants.

### 1.3.2.3 Planned workload sheet

**What it represents.** A resource-month matrix giving the man-days planned for each resource over the life of the project, together with the resulting forecast cost.

**Business purpose.** It is the workload plan : it expresses how the sold effort is expected to be consumed over time, and therefore what the project should cost.

**Key information and calculations.** With around four hundred formulas, this sheet computes the monthly and cumulative totals of planned man-days, the forecast cost  $\Sigma (\text{planned man-days} \times TCC)$ , and the *Estimate To Complete*  $ETC = \Sigma (\text{remaining man-days} \times TCC)$  as a running remainder.

**Derived PMS functionality.** The *Workload planning* module (Sprint 4), which stores the plan as a monthly time series per resource, and feeds the ETC used by the KPI engine.

### 1.3.2.4 Actual workload sheet

**What it represents.** The mirror of the previous sheet, filled with the effort actually consumed : the man-days really booked by each resource, month by month.

**Business purpose.** It measures reality against the plan. The gap between the two sheets is what reveals a project drifting.

**Key information and calculations.** It is the densest sheet of the workbook (more than

five hundred formulas). Effort may be captured in hours and normalised into man-days by `man-days = hours / 8`. The actual cost follows the same rule as the forecast,  $\Sigma (\text{consumed man-days} \times \text{TCC})$ , with monthly and cumulative totals. History must be preserved : an entry is never deleted.

**Derived PMS functionality.** The *Actual workload* module (Sprint 4) : submission restricted to assigned projects, validation by the project manager, preserved history, and the actual cost that feeds the EAC.

#### 1.3.2.5 Billing progress sheet

**What it represents.** The billing milestones of the contract, expressed as a percentage of the total, with their amount, their status and the monthly cumulative invoiced total.

**Business purpose.** It tracks the revenue side of the project : what has been invoiced, what has been collected, and what remains to be billed.

**Key information and calculations.** Each milestone amount is computed as `contract total percentage` ; the sheet maintains the invoiced cumulative total, the remaining amount, and the billing progress `invoiced cumulative / contract total`. A structural rule governs the sheet : the sum of milestone percentages may not exceed 100 %.

**Derived PMS functionality.** The *Billing* module (Sprint 6) : milestones with percentage, amount and lifecycle status, payments recorded against a milestone, and the automatic billing-progress indicator.

#### 1.3.2.6 Missions sheet

**What it represents.** The business trips carried out for the project, with their cost components : per diem, plane ticket, accommodation, transport and administrative stamp.

**Business purpose.** Travel costs are a real charge on the project margin ; they must be attached to the project rather than treated as a general overhead.

**Key information and calculations.** The total cost of a mission is the sum of its components, with the per diem obtained by multiplying the number of nights by a daily allowance converted at the applicable exchange rate.

**Derived PMS functionality.** The *Missions* module (Sprint 6) : a mission attached to a project and an employee, with multi-currency cost components and an automatically computed total.

#### 1.3.2.7 Project dashboard sheet

**What it represents.** The steering board of a single project : margins, man-days consumed against sold, key dates and the overall health of the project.

**Business purpose.** It is the sheet the project manager and the director actually look at during the monthly review. It answers one question : is this project still profitable ?

**Key information and calculations.** The dashboard does not compute the raw data ; it reads values produced by the other sheets through lookups and presents the margins and progress indicators. This separation between computation and presentation is precisely the design later adopted in PMS.

**Derived PMS functionality.** The *KPI dashboards* (Sprint 8) : a per-project indicator board, and its portfolio-level counterpart for the director.

#### 1.3.2.8 Statistics sheet — KPI evolution

**What it represents.** The real computation engine of the workbook : the month-by-month evolution of the financial indicators of the project.

**Business purpose.** It transforms the raw data of the other sheets into the earned-value indicators used to steer the project and to detect drift early.

**Key information and calculations.** This is the most formula-dense sheet of the workbook (more than six hundred formulas). It computes, per selected month, the production revenue **contract total EV %**, the estimate at completion **EAC = actual cost + ETC**, the net margin at **EAC Budget – EAC**, the accrued income (FAE) as the difference between production revenue and the invoiced total, and the drift in man-days between what was sold, consumed and remaining.

**Derived PMS functionality.** The *KPI engine* (Sprint 8). This sheet is the direct origin of the hybrid design retained in PMS : indicators recomputed on read from the live data, plus a frozen monthly snapshot that preserves the official state of each review. This snapshot is the digital equivalent of the monthly column of this sheet.

#### 1.3.2.9 Traceability towards the platform

Table 1.2 summarises the correspondence established during this preparation phase between the sheets analysed and the modules built.

| Excel sheet          | Business logic captured                                     | PMS module (sprint)                          |
|----------------------|-------------------------------------------------------------|----------------------------------------------|
| Fiche identification | Contractual and financial baseline, provisions, sold margin | Project management (S3), Internal quote (S7) |
| TCC                  | Daily loaded cost per resource and per year                 | Resources and cost rates (S2)                |

| Excel sheet            | Business logic captured                           | PMS module (sprint)    |
|------------------------|---------------------------------------------------|------------------------|
| Coût Prévisionnel      | Workload plan and forecast cost, ETC              | Workload planning (S4) |
| Coût réel Actualisé    | Consumed effort and actual cost                   | Actual workload (S4)   |
| Avancement_Facturation | Milestones in % of contract, billing progress     | Billing (S5)           |
| Missions               | Travel cost components                            | Missions (S5)          |
| Dashboard              | Per-project indicator board                       | KPI dashboards (S7)    |
| Statistiques           | Earned-value engine, EAC, margins, accrued income | KPI engine (S7)        |

**TABLEAU 1.2** : Traceability from the Excel workbook to the PMS modules

### 1.3.3 Existing market solutions

Before deciding to develop a platform, the question of adopting an existing tool has to be answered. Four widely deployed solutions were examined. The criteria are not general ones : they are taken from the workbook analysed above, because a replacement tool is only useful if it can carry the business logic the workbook already encodes.

**Microsoft Project** is the reference for scheduling. It handles task dependencies, critical paths and resource levelling well, and it does attach a cost to a resource. That cost, however, is a rate attached to the resource, not a rate attached to the resource *for a given year*, and the tool has no notion of a billing milestone expressed as a percentage of a contract, nor of revenue earned but not yet invoiced. It plans work ; it does not steer a contract.

**Jira** is the reference for agile planning, and its backlog and sprint management are considerably richer than what this project needed to build. It carries no financial model natively : costing requires marketplace applications such as Tempo, and even then the model is time-based rather than contract-based. Its permission schemes are powerful but operate on issues and projects rather than on individual financial fields, so the requirement that a developer must never see the margin of the project they work on is awkward to express.

**Odoo Project** comes closest, because it belongs to a suite that already contains invoicing and timesheets, so the link between effort and revenue exists. Its cost model nonetheless rests on a single hourly cost per employee, and its margin computation is Odoo's own. Adopting it would mean either accepting a financial model different from the one the company uses, or customising a full ERP, which

is a heavier commitment than developing a focused application.

**monday.com** offers flexible boards, formula columns and attractive dashboards. Its formulas are evaluated per board and do not compose across boards, which reproduces at a different scale the very fragility that makes the current workbook unreliable. It has no earned-value model.

### 1.3.4 Comparative synthesis

Table 1.3 compares the four tools against the requirements extracted from the workbook. The symbol ● marks native support, ○ support obtainable through configuration or an add-on, and – the absence of the capability.

| Requirement from the workbook                                  | MS Project | Jira | Odoo | monday | PMS |
|----------------------------------------------------------------|------------|------|------|--------|-----|
| Loaded cost rate per resource and per year                     | –          | –    | –    | –      | ●   |
| Billing milestones as a percentage of contract, capped at 100% | –          | –    | ○    | –      | ●   |
| Revenue earned but not yet invoiced                            | –          | –    | ○    | –      | ●   |
| Earned-value indicators (EV, EAC, ETC, margin)                 | ○          | –    | –    | –      | ●   |
| Multi-currency conversion at the business date                 | –          | –    | ○    | –      | ●   |
| Mission expenses decomposed by component                       | –          | –    | ○    | –      | ●   |
| Authorization configurable without redeployment                | ○          | ○    | ○    | –      | ●   |
| Agile planning : backlog and sprints                           | ○          | ●    | ●    | ●      | ●   |
| Scheduling and critical path                                   | ●          | ○    | ●    | ○      | –   |

**TABLEAU 1.3 :** Comparison of existing tools against the requirements of the reference workbook

The table should be read in both directions. The existing tools are strong exactly where the workbook is weak, in scheduling and in agile planning, and PMS makes no attempt to compete with

Microsoft Project on critical-path analysis. They are absent exactly where the workbook is strong, in the financial model that ties a loaded cost rate to a year, a milestone to a percentage of a contract, and an accrual to work already delivered.

That asymmetry is the justification for building rather than buying. The company did not keep the workbook because it was unaware of these tools; it kept it because none of them models the economics of a consulting firm working on fixed-price and time-and-material contracts in several currencies. The purpose of PMS is to carry that model reliably, not to reinvent project scheduling.

### 1.3.5 Critique of the existing system

While this workbook expresses a proven business logic, its medium, the spreadsheet, introduces structural limitations :

- **Dispersion** : one workbook per project, duplicated and liable to diverge from one version to the next, with no single source of truth.
- **Computation reliability** : the dependency on formulas and cross-tab references is fragile ; the actual file provided contains cells in error (`#REF!`, `#VALUE!`).
- **Financial visibility** : the absence of consolidation makes it difficult to read actual cost and margin at portfolio level.
- **Access control** : a spreadsheet cannot partition information finely ; it cannot, for instance, prevent a developer from accessing financial data.
- **Traceability** : the history of modifications and decisions is not reliably preserved.

## 1.4 Problem statement

The central question is therefore the following : *how can the company be equipped with a centralised, reliable and secure tool that automates the project financial logic currently carried by Excel, while remaining configurable enough to absorb the evolution of business rules without a rewrite ?*

Two requirements emerge from the study. The first is fidelity : the platform must preserve the richness of the existing business model, that is cost rates, workload plan, estimate at completion, margins and billing, rather than a simplified approximation of it, because that model is what the company actually uses to decide. The second is configurability : the authorization architecture must be dynamic and decoupled from roles, so that a change in who may see what is an administrative act rather than a release.

## 1.5 Proposed solution

The chosen solution is the development of a centralised web platform, **PMS** (*Project Management System*). It provides a single source of information and automates the operational and financial steering of projects. Its functional objectives are to :

- centralise project, resource and team data ;
- plan workload in man-days and record actual workload ;
- manage billing milestones, payments, contract amendments and missions ;
- support agile planning of the work through a product backlog and sprints ;
- automatically compute financial indicators, that is earned value, estimate at completion, margins, billing progress, remaining work and drift ;
- provide decision-support dashboards tailored to each user profile ;
- rest on a dynamic “Role → Permission → Module” access control.

## 1.6 Project management methodology

### 1.6.1 The Scrum framework

Scrum is an agile framework for developing products incrementally. Rather than specifying a system completely before building it, Scrum organises the work into short iterations called **sprints**, each producing an increment of the product that can be demonstrated and evaluated. Requirements are held in an ordered **product backlog**, from which the items to be built are drawn at the start of every sprint into a **sprint backlog**.

The framework rests on empirical process control : rather than predicting the whole system in advance, it inspects a working increment at regular intervals and adapts the remaining work accordingly.

### 1.6.2 Scrum roles

Scrum defines three accountabilities.

- The **Product Owner** is responsible for the value of the product. This role orders the product backlog, arbitrates priorities and accepts or rejects the increment.
- The **Scrum Master** is responsible for the process. This role ensures the framework is understood and applied, and removes the impediments that slow the work.

- The **Development Team** builds the increment. It is self-organising : it decides how to turn a backlog item into working software.

### 1.6.3 Scrum ceremonies

Four events structure a sprint, complemented by the continuous refinement of the backlog.

- **Sprint planning** opens the sprint. The team selects the backlog items it commits to and defines the sprint goal.
- The **daily scrum** is a short synchronisation in which the team reviews its progress towards the sprint goal and adjusts its plan.
- The **sprint review** presents the increment to the stakeholders and gathers their feedback.
- The **sprint retrospective** closes the sprint by examining the process itself and identifying an improvement to apply next.
- **Backlog refinement** runs continuously : items are clarified, split and re-ordered so that the next sprint can start from a ready backlog.

### 1.6.4 Adaptation to this project

Scrum assumes a team. This project was carried out by a single student developer, so the framework was applied in a deliberately reduced form, and it is worth stating plainly which parts were kept and which were not.

The **iterative and incremental principle was kept in full**, and it was the right choice for this domain. The PMS business model is deeply interdependent : the indicator engine consumes workload, billing, cost rates and the internal quote. Building it in a single pass would have meant discovering integration errors only at the end. Proceeding by increments allowed each functional block to be validated against the reference workbook before the next was built on top of it.

The roles were distributed as the situation allowed. The **industrial supervisor acted as product owner**, arbitrating priorities and validating each increment against the company's expectations. The **academic supervisor** followed the engineering and documentation quality. The author held the development role.

The ceremonies were adapted rather than performed formally. With one developer there is no daily synchronisation to hold, and no retrospective in the collective sense. What replaced the sprint review is more specific to this project and more demanding : **each increment was validated against the Excel workbook itself**, by reproducing on the platform a computation the workbook already

performed and comparing the two results. The workbook, being the company's real specification, served as the acceptance criterion.

Two distinct notions structure the remainder of this report and must not be confused. The **engineering phases**, that is business analysis, architecture design, database design and UML modelling, constitute the upstream study work and are presented in Chapters 2 and 3. The **development sprints** describe the progressive construction of the software, each delivering a functional increment, and are presented in Chapters 4 to 7. The distribution of the backlog across releases and sprints, together with the schedule, is established in Chapter 2.

## Conclusion

This chapter presented ST2i and the subject it assigned, then studied the process currently used to steer projects. The analysis of the reference workbook showed a proven business model carried by a fragile medium, and the comparison with the tools available on the market showed that none of them models that economics natively. From this follows the problem statement and the solution retained : a centralised platform that reproduces the financial model faithfully while making its authorization policy configurable. The next chapter specifies precisely what the system must do, for whom, and under which rules.

---

# REQUIREMENTS ANALYSIS AND SPECIFICATION

---

## Plan

|   |                                           |    |
|---|-------------------------------------------|----|
| 1 | Actor Identification . . . . .            | 18 |
| 2 | Functional Requirements . . . . .         | 18 |
| 3 | Non-Functional Requirements . . . . .     | 19 |
| 4 | Business Rules . . . . .                  | 20 |
| 5 | The Dynamic Authorization Model . . . . . | 21 |
| 6 | Global Use Case Diagram . . . . .         | 21 |
| 7 | Product Backlog . . . . .                 | 22 |
| 8 | Release and Sprint Planning . . . . .     | 25 |

## Introduction

This chapter formalises the business analysis of the PMS system. It identifies the actors, lists the functional and non-functional requirements, presents the business rules and the dynamic authorization model that structures the whole platform, and derives from them the global use case diagram. It closes with the product backlog and its distribution across the development sprints presented in the following chapters.

### 2.1 Actor Identification

The system rests on four main roles. Each role is defined as much by what it may do as by what it is explicitly prevented from doing, a distinction that is central to a platform handling sensitive financial data.

| Actor           | Mission                                                                                                              | Boundary (cannot)                                         |
|-----------------|----------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------|
| Administrator   | Technical and functional administration : users, roles, permissions, cost rates, configuration.                      | Steer projects ; access project financial data.           |
| Director        | Strategic governance : project creation, budgets, project manager assignment, portfolio supervision, executive KPIs. | Manage day-to-day operations (delegated, not forbidden).  |
| Project Manager | Main operational user : teams, workload plan, billing, missions, cost and profitability tracking.                    | Manage users, roles, cost rates and configuration.        |
| Developer       | Entry of actual workload ; consultation of own assignments and missions.                                             | Access any financial data (budget, margin, EAC, billing). |

**TABLEAU 2.1** : PMS actors and their boundaries

### 2.2 Functional Requirements

Functional requirements are organised by module. They synthesise the requirements of the SRS document (FR-001 to FR-053) and the use cases (UC-001 to UC-020).

| Module | Main functional requirements |
|--------|------------------------------|
|--------|------------------------------|

| Module                                                                          | Main functional requirements                                                                                                             |
|---------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| Authentication                                                                  | Login by e-mail and password, generation of JWT access and refresh tokens, mandatory password change on first login, logout.             |
| User management                                                                 | Creation by the administrator, automatic generation of a temporary password, modification, activation and deactivation, role assignment. |
| Roles and permissions                                                           | Management of roles, permissions and role-permission associations.                                                                       |
| Project management                                                              | Creation, modification, status change, budget definition, project manager assignment, portfolio consultation.                            |
| Team management                                                                 | Assignment and removal of developers, availability consultation, assignment history.                                                     |
| Cost rate (TCC) management                                                      | Entry of the loaded cost per resource and per year, history, daily cost computation.                                                     |
| Workload plan                                                                   | Monthly planning in man-days, planned totals, ETC and forecast computation.                                                              |
| Actual workload                                                                 | Entry of man-days consumed (assigned projects only), validation, history.                                                                |
| Billing                                                                         | Milestone creation (percentage, amount, dates), status tracking, progress computation.                                                   |
| Missions                                                                        | Mission creation, expense entry, automatic total cost computation.                                                                       |
| KPI and reporting                                                               | Financial indicator computation, project and portfolio dashboards, report generation.                                                    |
| Governance<br>(amendments,<br>risks, deliverables,<br>stakeholders,<br>changes) | Registers tracking the life of the project, in line with the Excel workbook.                                                             |

**TABLEAU 2.2** : Functional requirements by module

## 2.3 Non-Functional Requirements

- **Performance** : response in under two seconds for routine operations (NFR-001).

- **Security** : mandatory JWT authentication, hashed passwords (NFR-002).
- **Authorization** : permission-based access control (dynamic RBAC, NFR-003).
- **Maintainability** : clean, modular, feature-oriented architecture (NFR-006).
- **Scalability** : support for growth and future integration (ERP, HR, accounting) without rewriting the core (NFR-005, BR-065).
- **Auditability** : history preservation and logical deletion (NFR-008).
- **Usability** : intuitive and responsive interface (NFR-007).

## 2.4 Business Rules

The business rules come from the BRS document (65 rules, BR-001 to BR-065), enriched by the rules extracted from the Excel file. The most structuring ones are summarised below.

| Domain     | Main rules                                                                                                                                                                                                                                  |
|------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Users      | Creation reserved to the administrator; no self-registration; a single role per user; unique e-mail; temporary password then change on first login.                                                                                         |
| Projects   | Creation reserved to the director; exactly one director and one active project manager per project; status among {DRAFT, ACTIVE, ON_HOLD, COMPLETED, CANCELLED}; unique project code; strictly positive budget; start date before end date. |
| Teams      | A developer may belong to several projects; removal preserves history; an assignment records its author and date; inactive developers cannot be assigned.                                                                                   |
| Cost rates | Managed by the administrator; one value per resource and per year; historical value immutable after financial validation.                                                                                                                   |
| Workload   | Workload expressed in man-days, never negative; entry restricted to assigned projects; history preserved (no deletion).                                                                                                                     |
| Billing    | A milestone belongs to one project; status among {PENDING, INVOICED, PAID}; positive amount; the sum of milestone percentages is at most 100 %.                                                                                             |

| Domain           | Main rules                                                                                                                                   |
|------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| Missions         | A mission belongs to a project and an employee ; start date before end date ; total cost computed automatically.                             |
| KPI and security | Financial KPIs visible only to the director and the project manager ; automatic computation ; RBAC applied by permission ; hashed passwords. |

**TABLEAU 2.3** : Summary of business rules

## 2.5 The Dynamic Authorization Model

The central architectural requirement of the project is to **never couple a role directly to a module**. Such a coupling (“ROLE → MODULE”) would require modifying and redeploying the code every time the company decides that a given profile may access an additional function.

The model retained therefore inserts an intermediate level :

$$\mathbf{Role} \rightarrow \mathbf{Permission} \rightarrow \mathbf{Module}$$

A role holds a set of permissions ; each permission authorises a precise operation on a module. The role–permission matrix lives in the database, not in the source code : granting a new right becomes a data change, applied without redeployment. This principle governs the whole platform and is verified in Sprint 2 (Chapter 4).

## 2.6 Global Use Case Diagram

Figure 2.1 presents the global use case diagram of the platform. The four actors are associated with the business use cases falling within their perimeter, in strict accordance with the boundaries stated in Table 2.1 : the administrator holds no project use case, and the developer holds no financial use case.

The diagram is deliberately drawn at the level of the functional domain rather than the individual operation. “Manage projects” covers creation, configuration, the assignment of a project manager, closure and archiving ; “Manage project execution” covers the team, the workload plan and its validation, and the iterations. Decomposing each of these into its elementary operations would produce a diagram of some forty ellipses that no longer shows what the system does, only how it is implemented. The detailed operations appear in the use case diagram of the sprint that delivers them, in Chapters 4 to 7.

Any functionality requiring a prior identification points to the “Authenticate” use case through an *include* relation, which avoids repeating the same association on each actor. The “Freeze the monthly review” use case extends “Monitor performance indicators” through an *extend* relation : consulting the indicators is the nominal behaviour, and freezing them into a snapshot happens only at the monthly review.

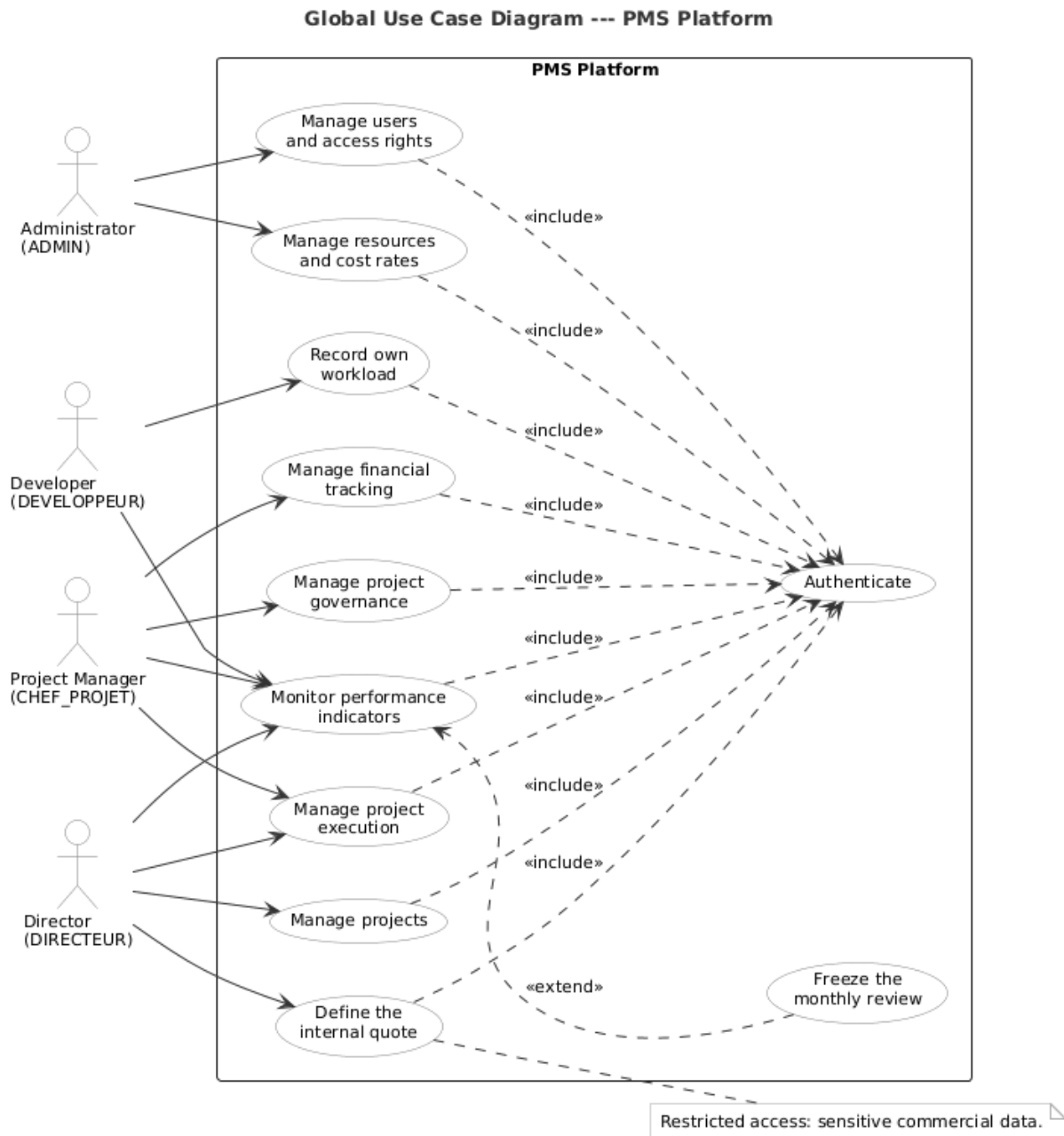


FIGURE 2.1 : Global use case diagram of the PMS platform

## 2.7 Product Backlog

The product backlog gathers the functional needs expressed as user stories. Each item is attached to the sprint that delivered it, which makes the incremental construction of the platform legible. Priority

is derived from technical dependency : a story is high priority when subsequent increments depend on it.

| <b>Id</b> | <b>User story</b>                                                                                                                                   | <b>Priority</b> | <b>Sprint</b> |
|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------------|-----------------|---------------|
| US-01     | As a user, I want to sign in with my e-mail and password so that I can access the platform.                                                         | High            | Sprint 1      |
| US-02     | As a user, I want to be forced to change my password on first login so that my account is secured.                                                  | High            | Sprint 1      |
| US-03     | As an administrator, I want a user's sessions to be revoked immediately when needed so that access can be cut off without waiting for token expiry. | High            | Sprint 1      |
| US-04     | As an administrator, I want to assign permissions to roles so that authorization evolves without redeploying the application.                       | High            | Sprint 2      |
| US-05     | As a user, I want the menu to show only what I am entitled to so that the interface matches my rights.                                              | High            | Sprint 2      |
| US-06     | As an administrator, I want to create, modify and deactivate user accounts so that I can manage platform access.                                    | High            | Sprint 2      |
| US-07     | As an administrator, I want to record resources and their yearly cost rate so that project costs can be computed.                                   | High            | Sprint 2      |
| US-08     | As a director, I want to create and configure a project so that it can be tracked on the platform.                                                  | High            | Sprint 3      |
| US-09     | As a director, I want a project to follow a controlled lifecycle so that no invalid status transition is possible.                                  | Medium          | Sprint 3      |
| US-10     | As a director, I want to assign a project manager to a project so that its operational steering is delegated.                                       | High            | Sprint 3      |
| US-11     | As a project manager, I want to build my project team so that I can allocate developers to it.                                                      | High            | Sprint 4      |
| US-12     | As a project manager, I want to plan the monthly workload so that I can forecast the effort required.                                               | High            | Sprint 4      |
| US-13     | As a developer, I want to submit the man-days I worked so that actual consumption is tracked.                                                       | High            | Sprint 4      |

| <b>Id</b> | <b>User story</b>                                                                                                                             | <b>Priority</b> | <b>Sprint</b> |
|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------|-----------------|---------------|
| US-14     | As a project manager, I want to validate the workload submitted so that only verified data feeds the indicators.                              | Medium          | Sprint 4      |
| US-15     | As a project manager, I want to define billing milestones so that invoicing follows contractual progress.                                     | High            | Sprint 6      |
| US-16     | As a project manager, I want to record payments received against a milestone so that collection is tracked.                                   | Medium          | Sprint 6      |
| US-17     | As a project manager, I want to record amendments so that the revised budget reflects contractual changes.                                    | Medium          | Sprint 6      |
| US-18     | As a project manager, I want to record missions and their expenses so that travel costs are included in the project cost.                     | Medium          | Sprint 6      |
| US-19     | As a project manager, I want to keep a risk register so that project risks are tracked and mitigated.                                         | Medium          | Sprint 7      |
| US-20     | As a project manager, I want to track deliverables and their status so that delivery progress is measurable.                                  | Medium          | Sprint 7      |
| US-21     | As a project manager, I want to record stakeholders and change requests so that project governance is documented.                             | Low             | Sprint 7      |
| US-22     | As a director, I want to define the internal quote so that the project has an economic baseline.                                              | High            | Sprint 8      |
| US-23     | As a project manager, I want the platform to compute the financial indicators automatically so that I no longer depend on the Excel workbook. | High            | Sprint 8      |
| US-24     | As a project manager, I want to freeze a monthly review so that the official state of the project is kept as history.                         | Medium          | Sprint 8      |
| US-25     | As a director, I want a portfolio dashboard so that I can supervise all projects at a glance.                                                 | High            | Sprint 8      |
| US-26     | As a user, I want a consistent and responsive interface so that the platform is comfortable to use daily.                                     | Medium          | Sprint 9      |
| US-27     | As a project manager, I want to organise the work into iterations so that delivery is planned in sprints.                                     | High            | Sprint 5      |

| <b>Id</b> | <b>User story</b>                                                                                               | <b>Priority</b> | <b>Sprint</b> |
|-----------|-----------------------------------------------------------------------------------------------------------------|-----------------|---------------|
| US-28     | As a project manager, I want a prioritised and estimated product backlog so that the remaining work is visible. | High            | Sprint 5      |
| US-29     | As a developer, I want to follow the current iteration on a board so that I know what is in progress.           | Medium          | Sprint 5      |

**TABLEAU 2.4** : Product backlog of the PMS platform

## 2.8 Release and Sprint Planning

The backlog is distributed across four releases grouping ten sprints. Each release brings the product to a coherent level of maturity : R1 makes the platform usable and secure, R2 opens operational steering, R3 delivers the financial and governance value, and R4 consolidates the whole.

| <b>Release</b>  | <b>Sprint</b> | <b>Scope</b>                             | <b>User stories</b>             |
|-----------------|---------------|------------------------------------------|---------------------------------|
| R1 — Foundation | Sprint 1      | Technical foundation, authentication     | US-01 to US-03                  |
|                 | Sprint 2      | Dynamic RBAC, users, resources           | US-04 to US-07                  |
| R2 — Operations | Sprint 3      | Projects and lifecycle                   | US-08 to US-10                  |
|                 | Sprint 4      | Teams and workload                       | US-11 to US-14                  |
|                 | Sprint 5      | Agile planning : backlog and iterations  | US-27 to US-29                  |
| R3 — Finance    | Sprint 6      | Billing and missions                     | US-15 to US-18                  |
|                 | Sprint 7      | Governance                               | US-19 to US-21                  |
|                 | Sprint 8      | Internal quote and KPI engine            | US-22 to US-25                  |
| R4 — Delivery   | Sprint 9      | Consolidation, tests, deployment         | US-26                           |
|                 | Sprint 10     | Containerization and continuous delivery | Technical sprint, no user story |

**TABLEAU 2.5** : Distribution of the backlog across releases and sprints

### 2.8.1 Schedule

The internship ran from 5 January to 30 August 2026, that is thirty-four weeks. Six weeks were devoted to the preparation phase, which analyses the existing Excel process before any code is written, and one week to the final assembly of the report. The remaining twenty-seven weeks were divided into ten sprints. Seven of them last three weeks ; the three shorter ones cover the smaller perimeters, namely agile planning, interface consolidation and the delivery chain.

| Phase               | Start       | End         | Weeks |
|---------------------|-------------|-------------|-------|
| Preparation phase   | 5 January   | 13 February | 6     |
| Sprint 1            | 16 February | 6 March     | 3     |
| Sprint 2            | 9 March     | 27 March    | 3     |
| Sprint 3            | 30 March    | 17 April    | 3     |
| Sprint 4            | 20 April    | 8 May       | 3     |
| Sprint 5            | 11 May      | 22 May      | 2     |
| Sprint 6            | 25 May      | 12 June     | 3     |
| Sprint 7            | 15 June     | 3 July      | 3     |
| Sprint 8            | 6 July      | 24 July     | 3     |
| Sprint 9            | 27 July     | 7 August    | 2     |
| Sprint 10           | 10 August   | 21 August   | 2     |
| Report finalisation | 24 August   | 28 August   | 1     |

**TABLEAU 2.6 :** Schedule of the preparation phase and the ten sprints

## Conclusion

This chapter established what the platform must do, for whom, and under which rules, and translated those requirements into a product backlog distributed across the development sprints. The next chapter presents the architecture and the design models, the data model and the database schema, on which this construction rests.

---

# SYSTEM ARCHITECTURE AND DESIGN

---

## Plan

|   |                                          |    |
|---|------------------------------------------|----|
| 1 | Architectural Style . . . . .            | 28 |
| 2 | Cross-Cutting Design Decisions . . . . . | 30 |
| 3 | Domain Model . . . . .                   | 31 |
| 4 | Database Design . . . . .                | 32 |
| 5 | Deployment Architecture . . . . .        | 34 |
| 6 | Technical Environment . . . . .          | 34 |

## Introduction

This chapter translates the requirements of Chapter 2 into a technical design. It presents the architectural style retained, the layered organisation of the application, the cross-cutting design decisions that structure every module, the domain model, and the relational schema that supports it. These choices are made once here and are then applied uniformly by the ten sprints of Chapters 4 to 7.

### 3.1 Architectural Style

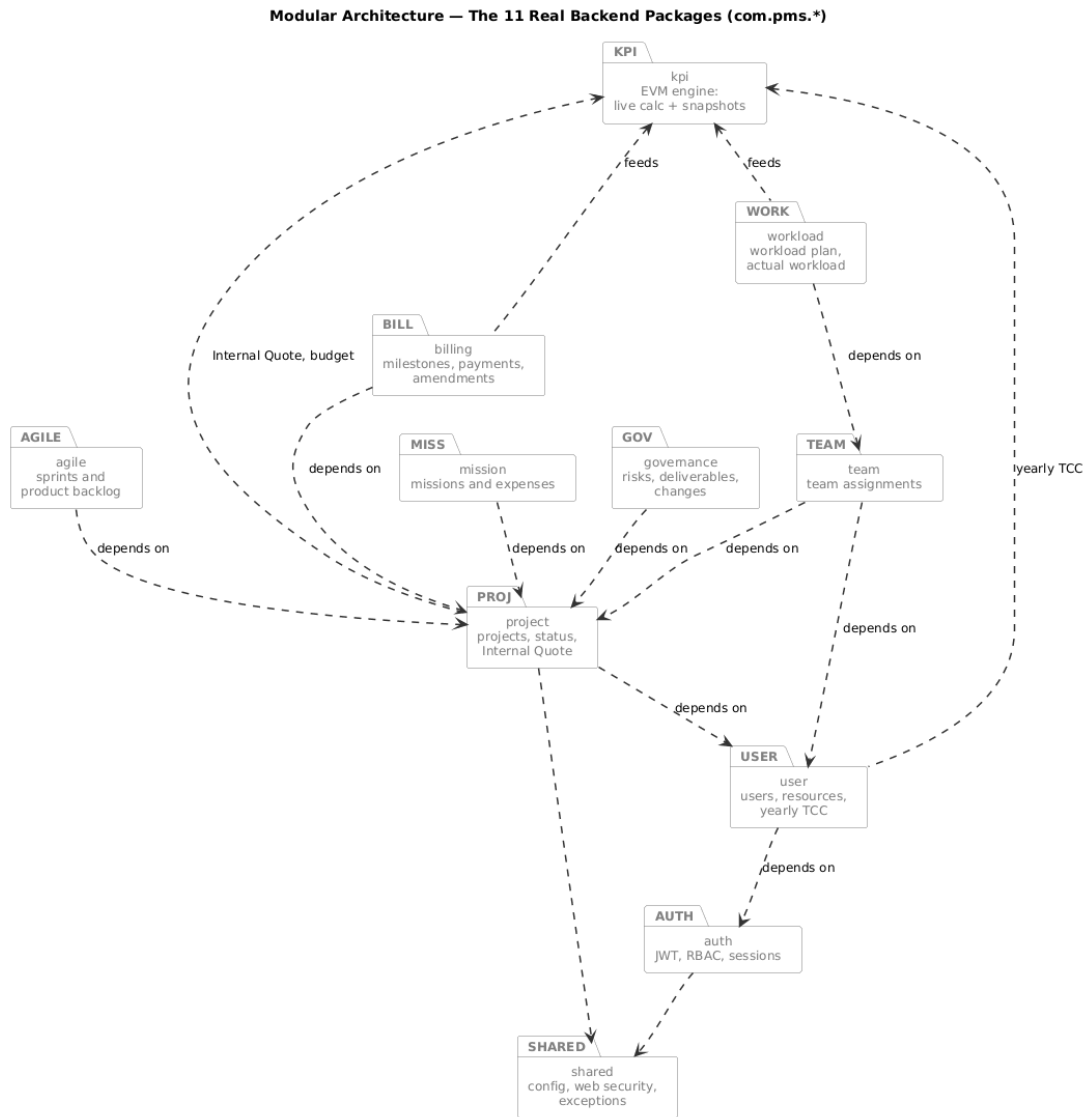
#### 3.1.1 A Three-Tier Modular Monolith

The platform follows a classical **three-tier** organisation (presentation, application, data), deployed as a **modular monolith** : a single deployable artefact whose code is partitioned into independent functional modules.

This choice deserves justification, since a microservice architecture is often the default expectation for a management platform. Three arguments led to the monolith :

- **Transactional consistency.** The PMS domain is strongly coupled : computing an indicator reads workload, billing, cost rates and the internal quote at once. Keeping these aggregates in a single transactional boundary avoids distributed transactions for no functional gain.
- **Deployment scale.** The platform targets the internal use of one company. The operational cost of a distributed architecture would not be repaid by the load.
- **Reversibility.** Partitioning the code into modules with explicit boundaries preserves the option of extracting a module into a separate service later, should the need arise.

Figure 3.1 shows the ten functional packages and their dependencies.



**FIGURE 3.1 :** Functional modules and their main dependencies

The dependency graph is deliberately acyclic and converges towards the `kpi` module, which consumes the others without being consumed by any : the indicator engine is the end of the chain, never a prerequisite.

### 3.1.2 Layered Organisation

Each module exposes the same internal layering :

- **Controller** — REST entry point, request validation, authorization annotation.
- **Service** — business logic, transaction boundary, rule enforcement.
- **Repository** — persistence access, implicit filtering of logically deleted rows.
- **Domain** — entities and enumerations carrying the business invariants.

Conversions between entities and data transfer objects are handled by generated mappers, so that persistence structures are never exposed directly by the API.

## 3.2 Cross-Cutting Design Decisions

### 3.2.1 Authentication and Session Revocation

Authentication rests on a short-lived JWT access token paired with an opaque refresh token stored in an `HttpOnly` cookie. Revocation does not rely on a blacklist but on a version counter carried by the token and compared on every request, which allows a session to be cut off instantly without distributed state. The mechanism is detailed in Sprint 1.

### 3.2.2 Dynamic Authorization

The “Role  $\rightarrow$  Permission  $\rightarrow$  Module” model presented in Section 2.5 is enforced by annotating every endpoint with the permission code it requires. No role name appears in application code. The role–permission matrix lives in the database, which makes the authorization policy configurable data. This is the single most structuring decision of the project and is validated in Sprint 2.

### 3.2.3 A Hybrid Indicator Engine

Financial indicators could be either recomputed on demand or persisted at each change. The design retained is **hybrid** :

- indicators are **recomputed on read** from live data, which guarantees that a dashboard never displays a stale figure ;
- a **monthly snapshot** freezes the official state of each review, which preserves the history required to plot the evolution of a project.

This mirrors the reference workbook, where the dashboard reads live values while the statistics sheet keeps one column per month. Amounts derived from the internal quote are always computed at read time and never stored, so that no commercially sensitive value is duplicated in the database.

### 3.2.4 Multi-Currency Handling

Projects may be contracted in a currency different from the reporting currency. Every amount therefore carries its currency, and conversions use a **dated rate** : each conversion applies the rate at the business date of the amount : the invoice date for billing, the month for monthly costs, the mission

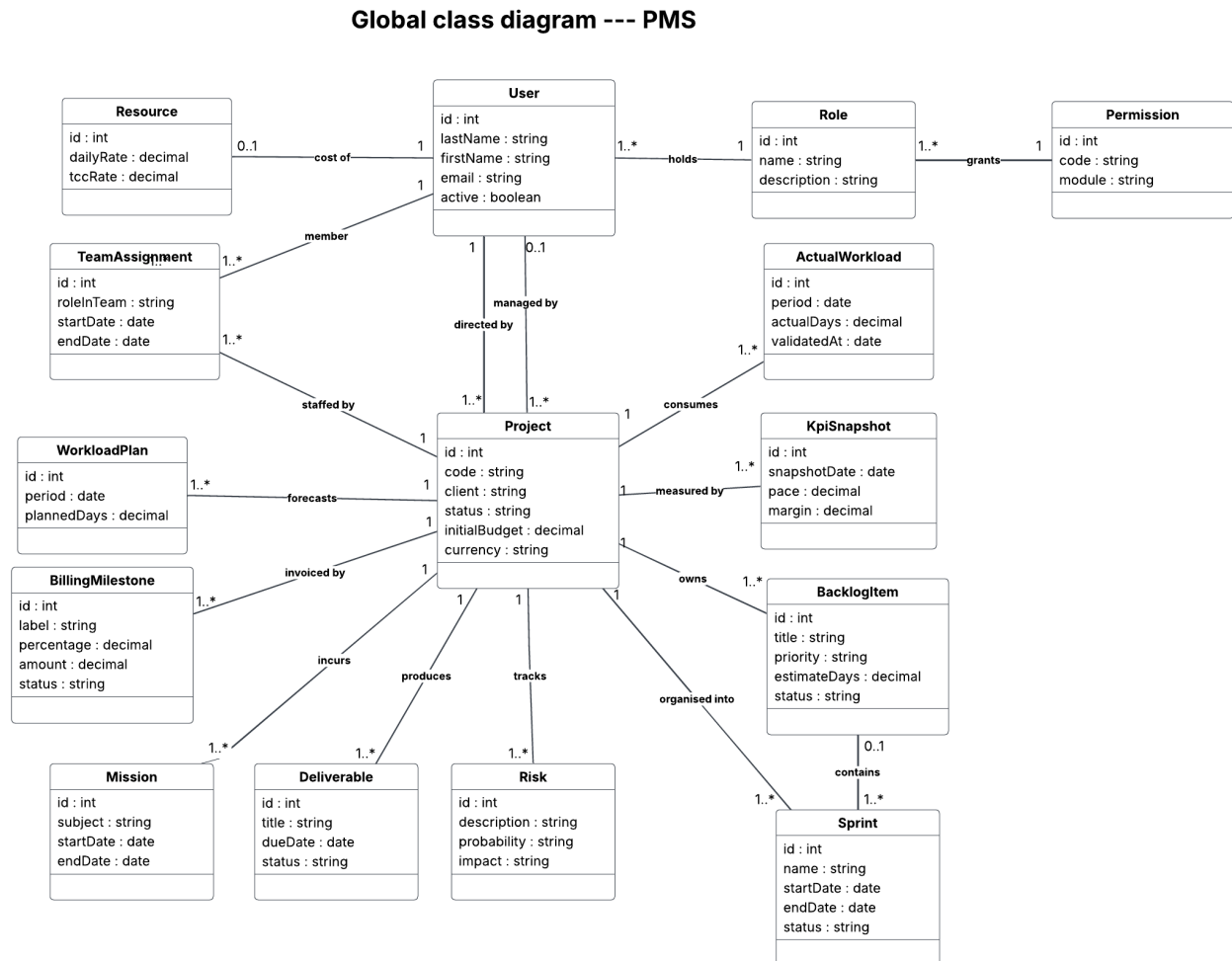
date for travel. Mission cost components are converted before summation, so that currencies are never mixed within a total.

### 3.2.5 Auditability and Logical Deletion

Every business table carries creation and modification metadata and an `active / deleted_at` pair. No row is physically removed : a deletion is a state change. This satisfies the auditability requirement and preserves the history on which the indicators depend.

## 3.3 Domain Model

Figure 3.2 gives the overall map of the domain. The project is the aggregation root : most business entities exist only within a project and are composed by it, while users, roles, permissions and resources form an independent reference core.

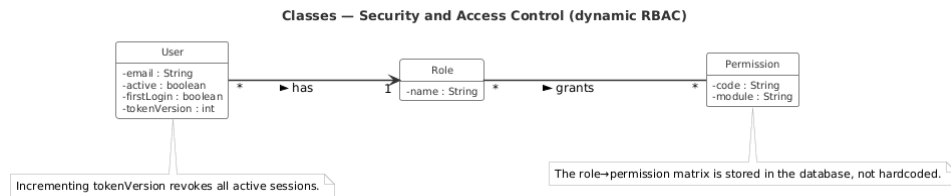


**FIGURE 3.2 :** Domain model — principal classes and their associations

One reading of the diagram deserves to be made explicit, because it is easy to expect the opposite. The four roles of the platform are not subclasses of **User** : they are rows of the **Role** class. A

user holds a role, and a role grants permissions, so the authorization policy is data that an administrator edits rather than a hierarchy compiled into the application. Modelling the roles as types would have contradicted that, and would have made every change of policy a new version of the software.

The diagram stays at the level of the principal classes and their key attributes. Each sub-domain is detailed by its own class diagram, presented in the sprint that builds it : security in Sprint 2 (Figure 3.3), projects and teams in Sprint 3, agile planning in Sprint 5, financial management in Sprint 6, governance in Sprint 7, and the indicator engine in Sprint 8.



**FIGURE 3.3 :** Class diagram — security and access control

## 3.4 Database Design

### 3.4.1 Modelling Conventions

The schema targets PostgreSQL and follows uniform conventions :

- tables in **snake\_case**, plural; identity primary key **id** ;
- foreign keys named **<entity>\_id** ; timestamps in **timestamptz** ;
- amounts stored as **numeric(18,3)** with an explicit currency column, never as floating point, to avoid rounding drift on financial aggregates ;
- months stored as the **first day of the month**, making the workload a true time series ;
- enumerations expressed as **varchar** constrained by **CHECK** ;
- audit columns and logical deletion on every business table.

### 3.4.2 Logical Model

The core of the schema is the dynamic access control : **users** references **roles**, and **roles** relates to **permissions** through the **role\_permissions** association. A project references its director and groups its child entities : project manager assignment (a single active one at a time), team assignments, workload plan, actual workload, billing milestones, missions, governance registers and KPI snapshots. A resource carries one cost rate per year and may be linked to a user account.

| Domain          | Main tables                                         | Module                |
|-----------------|-----------------------------------------------------|-----------------------|
| Security / RBAC | users, roles, permissions, role_permissions         | Authentication & RBAC |
| Reference data  | currencies, exchange_rates, parameters, risk scales | Configuration         |
| Resources       | resources, tcc                                      | Cost rate management  |
| Projects        | projects, project_manager_assignments               | Project management    |
| Teams           | team_assignments, stakeholders                      | Teams                 |
| Workload        | workload_plan, actual_workload                      | Planning / Actuals    |
| Billing         | billing_milestones, payments, avenants              | Billing               |
| Missions        | missions                                            | Mission management    |
| Governance      | risks, deliverables, change_requests, actions       | Governance            |
| Indicators      | kpi_snapshots                                       | KPI & Reporting       |

TABLEAU 3.1 : Catalogue of entities by domain

### 3.4.3 Monthly Time Series and Indicator Snapshots

The Excel workbook is a resource month grid. Rather than reproducing twenty-five month columns, the model is **normalised** : one row per (project, resource, month) in **workload\_plan** and **actual\_workload**. Adding a month becomes inserting a row rather than altering the schema.

The **kpi\_snapshots** table stores one snapshot per (project, month), which yields both the history needed for evolution curves and fast reads for dashboards. It deliberately separates the **labour cost** (man-days cost rate) from **other costs** (missions and expenses), the total aggregating the two. The original workbook blurred that distinction, which matters when analysing why a project drifts.

### 3.4.4 Reference Data and Migration Strategy

A seed dataset creates the four roles, the full permission catalogue, the default role–permission matrix, the currencies, the business parameters, the risk scales and a bootstrap administrator.

The schema is managed exclusively by **Flyway** versioned SQL migrations, with the ORM in **validate** mode : the entities must match the schema, never generate it. The migrations constitute the

living documentation of the database and make every schema change reviewable and reproducible.

### 3.5 Deployment Architecture

Figure 3.4 shows the target deployment : a browser running the single-page application, an application server exposing the REST API, and a database server. The whole platform is a single deployable artefact, consistent with the modular monolith decision.

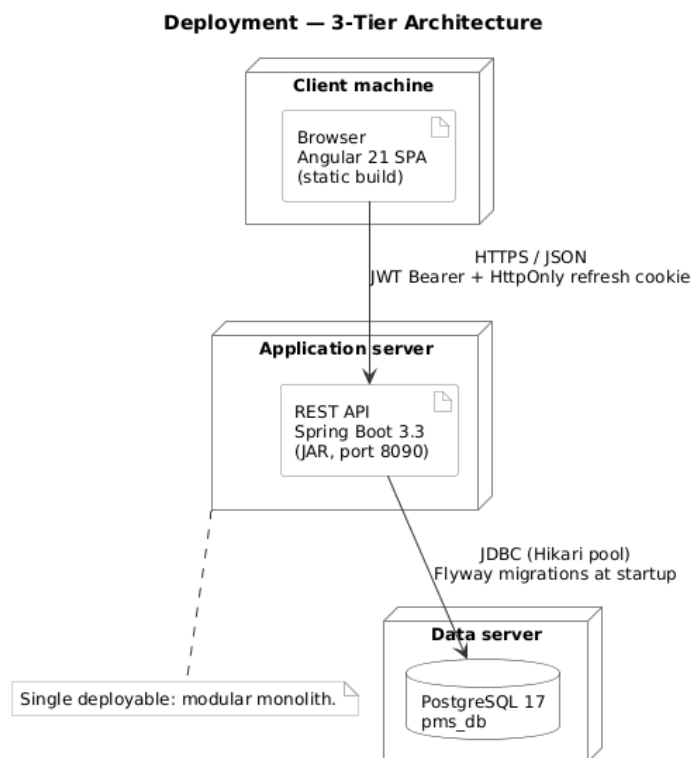


FIGURE 3.4 : Deployment architecture

### 3.6 Technical Environment

| Layer     | Technologies                                                 |
|-----------|--------------------------------------------------------------|
| Backend   | Java, Spring Boot, Spring Security, JPA/Hibernate, MapStruct |
| Database  | PostgreSQL, Flyway migrations                                |
| Frontend  | Angular, TypeScript, standalone components and signals       |
| Security  | JWT access and refresh tokens, BCrypt password hashing       |
| Modelling | PlantUML for the UML diagrams, versioned as source           |

TABLEAU 3.2 : Technical environment of the project

## Conclusion

The architecture retained, a three-tier modular monolith with dynamic authorization, a hybrid indicator engine and a normalised time-series schema, answers the requirements of Chapter 2 while remaining proportionate to the scale of the target deployment. These decisions are now applied : the following chapters present the progressive construction of the platform, release by release and sprint by sprint.

# AUTHENTICATION AND ACCESS CONTROL

---

## Plan

|   |                                                                  |    |
|---|------------------------------------------------------------------|----|
| 1 | Release 1 backlog . . . . .                                      | 37 |
| 2 | Sprint 1 — Authentication and technical foundation . . . . .     | 38 |
| 3 | Sprint 2 — Dynamic access control, users and resources . . . . . | 40 |

## Introduction

This chapter presents the first release of the PMS platform. Its purpose is to make the application usable and secure before any business functionality is built on top of it. It gathers two sprints : Sprint 1 establishes the technical foundation and the authentication chain, Sprint 2 delivers the dynamic authorization model together with the management of users and resources.

At the end of this release the platform authenticates its users, applies a configurable authorization policy, and holds the reference data, that is users, resources and cost rates, which the operational modules of the next release consume.

### 4.1 Release 1 backlog

The first release realises the seven user stories of the product backlog (Table 2.4) that concern access to the platform. Because the author is the sole developer, each story is expanded into the technical work it required.

| Story | Need                                                   | Technical work                                                                                                                                                          | Sprint |
|-------|--------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| US-01 | Sign in with an e-mail and a password                  | Authentication service, BCrypt verification, JWT issuance signed with HMAC-SHA512, refresh cookie, token rotation                                                       | 1      |
| US-02 | Be forced to change an imposed password on first login | <code>first_login</code> flag, dedicated filter restricting the token to the password-change endpoint                                                                   | 1      |
| US-03 | Revoke a user's sessions immediately                   | Token version persisted per user, embedded in each token and compared on every request                                                                                  | 1      |
| US-04 | Assign permissions to roles without redeploying        | <code>roles</code> , <code>permissions</code> and <code>role_permissions</code> tables, permissions carried in the token and converted into Spring Security authorities | 2      |
| US-05 | See only the menu entries the account is entitled to   | Context endpoint returning the held permissions, navigation built from that response                                                                                    | 2      |

| Story | Need                                        | Technical work                                                                               | Sprint |
|-------|---------------------------------------------|----------------------------------------------------------------------------------------------|--------|
| US-06 | Create, modify and deactivate user accounts | User module, creation with a temporary password, activation state, role assignment           | 2      |
| US-07 | Record resources and their yearly cost rate | Resource entity separate from the account, loaded cost rate stored per resource and per year | 2      |

**TABLEAU 4.1** : Backlog of release 1

## 4.2 Sprint 1 — Authentication and technical foundation

### 4.2.1 Sprint 1 use case diagram

The sprint covers the three use cases that every user must traverse before reaching a business screen : authenticating, changing an imposed password, and terminating a session. Every other use case of the platform includes authentication, which is why this sprint precedes all the others.

### 4.2.2 Sprint 1 analysis and design

#### 4.2.2.1 Sequence of the “Authenticate” use case

The nominal scenario involves the browser, the authentication controller, the authentication service, the token service and the user repository. It shows where the password is verified, where the two tokens are produced, and why only one of them is written to a cookie.

#### 4.2.2.2 Sequence of the “Change the password on first login” use case

An account created by the administrator carries a temporary password. The scenario shows how the token issued at the first connection is restricted, how any business call is refused while the flag remains set, and how a full token is issued once the password has been changed.

#### 4.2.2.3 Sequence of the “Revoke a session” use case

Revocation is the scenario that justifies the design of the token. It shows the increment of the version, the invalidation of the cache entry, and the rejection of a request presenting a token issued before the increment.

#### 4.2.2.4 Design decisions

Three decisions taken in this sprint constrain the whole platform and are therefore justified here rather than in the realisation.

**A modular monolith rather than services.** The code is organised into independent functional packages, each with its own Controller → Service → Repository → Domain layering. A single deployable artefact keeps transactions local and avoids distributed consistency, which matters because the financial computations of release 3 read from several modules within one transaction. The package boundaries preserve the option of extraction later.

**Two tokens rather than one.** A short-lived access token held in memory is paired with a refresh token confined to an `HttpOnly` cookie. A single long-lived token stored in the browser would be readable by any injected script ; splitting them means the credential able to mint new sessions is never reachable from JavaScript. The refresh token is additionally rotated on every use, so a stolen cookie stops working as soon as the legitimate holder refreshes.

**Revocation by version rather than by blacklist.** A JWT is self-contained and cannot be withdrawn once issued. A server-side blacklist would reintroduce the shared state that the stateless design avoids. Storing a single integer per user and embedding it in each token achieves the same result : incrementing it invalidates every session of that identity at once, at the cost of one integer comparison per request.

### 4.2.3 Sprint 1 realization

#### 4.2.3.1 Modular monolith structure

The modules are `auth`, `user`, `project`, `team`, `workload`, `agile`, `kpi`, `billing`, `mission`, `governance` and `shared`. Each subsequent sprint adds its module without modifying the previous ones.

#### 4.2.3.2 Base entity and automatic auditing

All business entities inherit from `BaseEntity`, which centralises the auto-generated primary key, the `createdAt` and `updatedAt` timestamps managed by `@PrePersist` and `@PreUpdate`, and the `deleted` marker implementing logical deletion. No data is ever physically erased, and every repository implicitly filters on `deleted = false`. This satisfies the auditability requirement (NFR-008).

#### 4.2.3.3 Schema management through Flyway

The database structure is managed exclusively by Flyway. Hibernate never emits implicit DDL : the property `spring.jpa.hibernate.ddl-auto` is set to `validate`, so a divergence between the entities

and the schema stops the application at startup rather than corrupting data at runtime. The versioned migrations are idempotent and constitute the living documentation of the schema.

#### 4.2.3.4 Token issuance and rotation

On `POST /api/auth/login` the authentication service verifies the e-mail, compares the supplied password with the BCrypt hash, checks that the account is active, then issues a short-lived JWT access token signed with HMAC-SHA512 carrying the e-mail, the token version and the list of permission codes, together with a refresh token placed in the protected cookie.

`POST /api/auth/refresh` exchanges a valid cookie for a new access token and a new refresh cookie ; the previous one is rejected from that moment on.

#### 4.2.3.5 Immediate revocation by token version

Revocation rests on a `token_version` field persisted in the `users` table and embedded in every issued token. On each request the `JwtAuthenticationFilter` compares the version carried by the token with the value read from an in-memory cache ; if they differ the request is rejected with `401 Unauthorized`. Incrementing the field therefore closes every active session of that identity instantly, which is what happens on logout and on password change, the latter also terminating sessions opened on other devices.

#### 4.2.3.6 Protection against repeated failures

`LoginAttemptTracker` counts consecutive failures per identity and temporarily refuses further attempts, answering `429 Too Many Requests`. Password guessing is therefore bounded without locking a legitimate account permanently.

#### 4.2.3.7 Forced password change on first login

When an account is created the administrator sets a temporary password and `first_login` is set to `true`. The token issued at the first connection carries a restricted right enforced by `FirstLoginFilter` : only `POST /api/auth/change-password` is authorised. Once the password is changed, `first_login` becomes `false` and an unrestricted token is issued.

### 4.3 Sprint 2 — Dynamic access control, users and resources

#### 4.3.1 Sprint 2 use case diagram

The sprint covers the administration use cases : assigning permissions to a role, managing user accounts, and recording resources with their yearly cost rate. All of them are reserved to the

administrator, and all of them include the authentication delivered by sprint 1.

### 4.3.2 Sprint 2 analysis and design

#### 4.3.2.1 Model of the authorization policy

The authorization model is the structuring design of the platform and deserves its own class diagram. A role aggregates permissions, a permission belongs to a functional module, and a user holds one active role. Nothing in this model is compiled into the application : the association between a role and its permissions is a table, so the policy is data.

#### 4.3.2.2 Sequence of the “Grant a permission to a role” use case

This scenario is the proof that the policy is configurable. It shows the administrator adding a permission to a role, the association being persisted, and the affected users obtaining the new right on their next token without any redeployment.

#### 4.3.2.3 Design decisions

**Permissions rather than roles in the code.** No role name is ever tested in the application. Only permission codes are, through annotations of the form `@PreAuthorize("hasAuthority('CREATE_PROJECT')")`. Testing a role would hard-code the policy into the source and make every change a release ; testing a permission leaves the policy in the database.

**A resource is not an account.** A resource represents a cost-bearing capacity and may exist without a platform account, since the workbook records people who never connect to any system. Merging the two notions would have made it impossible to cost a contributor who has no login.

**A cost rate per year, not per resource.** The loaded cost rate is recorded per resource *and per year*, so that the cost of a man-day uses the rate of the year in which the effort was booked. This requirement comes directly from the workbook, where the 2024 and 2025 rates of the same person differ.

### 4.3.3 Sprint 2 realization

#### 4.3.3.1 Permission, role and user model

The `role_permissions` table associates a set of permissions with a role. A user holds a single active role, but that role can be reconfigured at any time by the administrator without redeployment. A dedicated migration initialises the four default roles, Administrator, Director, Project Manager and Developer, with eighteen permission codes spread over seven functional modules.

#### 4.3.3.2 Propagation to Spring Security

The `JwtAuthenticationFilter` reads the permission codes from the token and converts them into `SimpleGrantedAuthority` objects, after which every REST entry point is protected by its `@PreAuthorize` annotation.

#### 4.3.3.3 Dynamic menu

The context endpoint returns, for the authenticated user, the list of their permissions together with their profile information. The Angular frontend builds the navigation menu from that response : an item is displayed only if the user holds the required permission code. The interface therefore reflects the authorization policy automatically, with no duplicated rule on the client side.

#### 4.3.3.4 Users and resources

The `user` module manages accounts, that is creation by the administrator with a temporary password, modification, activation and deactivation, and role assignment, together with the human resources and their daily and loaded cost rates. Conversions between entities and data transfer objects are handled by MapStruct mappers.

## Conclusion

Release 1 established the foundations on which the entire platform rests : a modular application skeleton with auditing and controlled migrations, a robust authentication chain, and above all a dynamic authorization model that makes the security policy configurable data rather than compiled code. The platform now knows not only who the user is but what that user may do, and it holds the reference data the business modules consume. The next release exploits this foundation to deliver the operational management of projects and workload.

# PROJECT AND OPERATIONAL MANAGEMENT

---

## Plan

|   |                                                       |    |
|---|-------------------------------------------------------|----|
| 1 | Release 2 backlog . . . . .                           | 44 |
| 2 | Sprint 3 — Project management and lifecycle . . . . . | 45 |
| 3 | Sprint 4 — Team assignment and workload . . . . .     | 48 |
| 4 | Sprint 5 — Agile planning . . . . .                   | 50 |

## Introduction

Release 1 delivered a secured and administrable platform, but one that managed no project. This release opens operational steering. Sprint 3 introduces the central entity of the whole domain, the project, with its controlled lifecycle and its budget. Sprint 4 populates it with a team and with the workload data on which every later indicator depends. Sprint 5 adds the agile planning of that work, through a product backlog and iterations.

At the end of this release a project can be created, staffed, planned, its actual consumption recorded and its remaining work organised into sprints : everything the financial engine of Release 3 will need as input.

### 5.1 Release 2 backlog

The second release realises the user stories of the product backlog (Table 2.4) that concern operational steering. As in the previous release, each story is expanded into the technical work it required.

| Story | Need                                                         | Technical work                                                                                         | Sprint |
|-------|--------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|--------|
| US-08 | Create a project with its contractual and financial identity | Project entity reproducing the identification sheet, effective budget derived from amendments          | 3      |
| US-09 | Prevent a project from reaching an inconsistent state        | Status enumeration carrying its own legal transitions, refusal with 422 in the service layer           | 3      |
| US-10 | Assign a project manager and keep the history                | Assignment closed rather than overwritten, single active assignment enforced by a partial unique index | 3      |
| US-11 | Build the project team                                       | Team assignment with a validity period, permission and project scope both checked                      | 4      |

| Story | Need                                                    | Technical work                                                                                                   | Sprint |
|-------|---------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|--------|
| US-12 | Plan the monthly workload per resource                  | Workload plan stored as a monthly time series, one row per project, resource and month                           | 4      |
| US-13 | Record the effort actually consumed                     | Actual workload restricted to assigned projects, month normalised to its first day, uniqueness enforced by index | 4      |
| US-14 | Validate a period and freeze it                         | Validation by the project manager, immutability of a validated month                                             | 4      |
| US-27 | Organise the work into iterations                       | Sprint entity with goal, dates and lifecycle, scoped to a project                                                | 5      |
| US-28 | Maintain a prioritised and estimated product backlog    | Backlog item with priority and an estimate in man-days, unattached to any sprint until committed                 | 5      |
| US-29 | Follow the progress of the current iteration on a board | Board columns derived from the item status, move endpoint reassigning sprint and column together                 | 5      |

TABLEAU 5.1 : Backlog of release 2

## 5.2 Sprint 3 — Project management and lifecycle

### 5.2.1 Sprint 3 use case diagram

The sprint covers the use cases through which a project comes into existence and changes state : creating and configuring a project, changing its status, assigning its manager, and archiving it once completed. All of them are reserved to the director, except archiving which the project manager may also perform.

## 5.2.2 Sprint 3 analysis and design

### 5.2.2.1 Model of the project and its team

The **project** module reproduces the identification sheet of the reference workbook : project code, client, funder, business model, engagement type (fixed price or time & material), project currency with its contractual exchange rate, initial budget, sold workload and provisions.

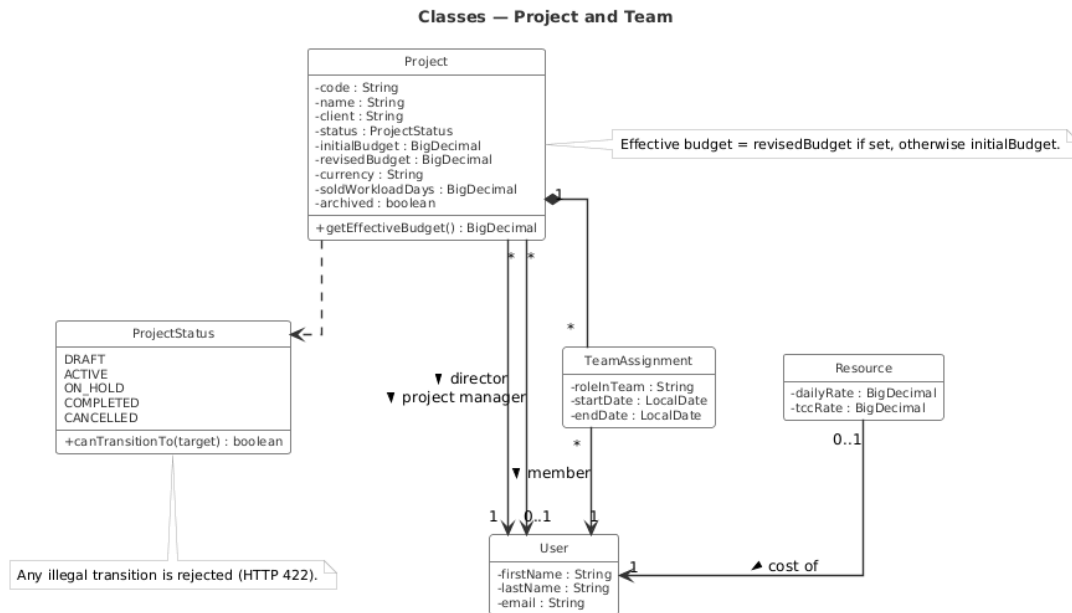


FIGURE 5.1 : Class diagram — projects and teams

### 5.2.2.2 The project lifecycle

A project moves through five states : DRAFT, ACTIVE, ON\_HOLD, COMPLETED and CANCELLED. Not every transition is legal : a completed project cannot return to draft, and a cancelled project is terminal.

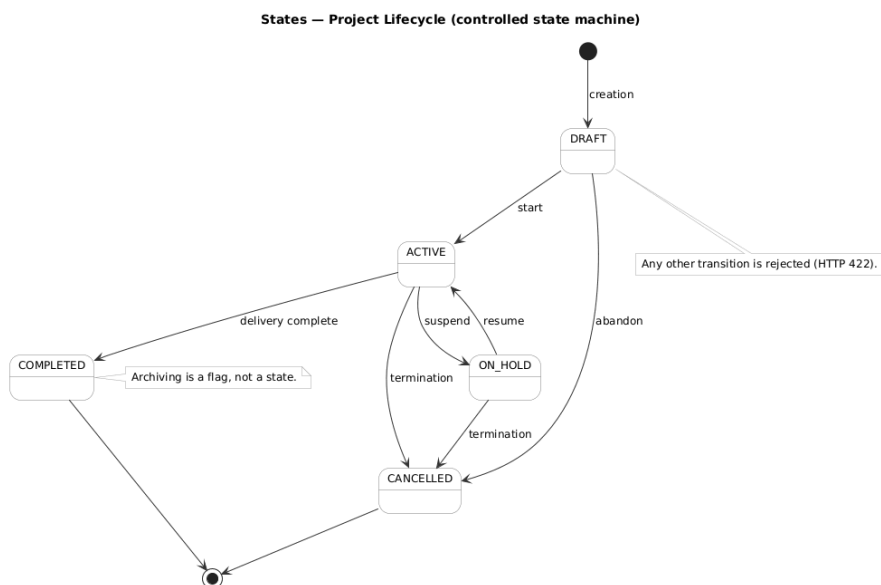
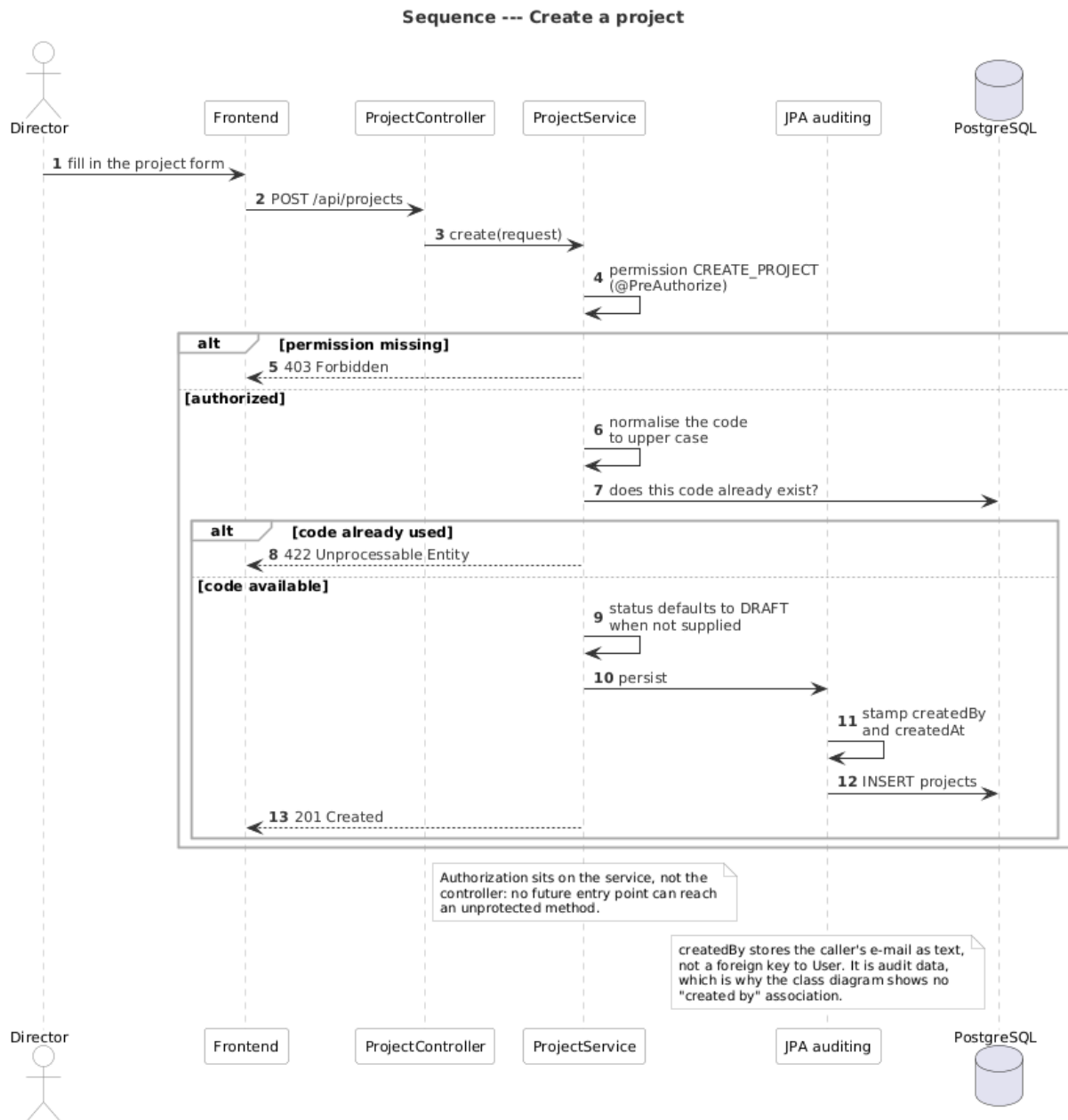


FIGURE 5.2 : State diagram — project lifecycle

### 5.2.2.3 Sequence of the “Create a project” use case

Creating a project is the operation that opens the whole domain, and its scenario shows both who may perform it and what the platform verifies before accepting it : the caller must hold the `CREATE_PROJECT` permission, the project code is normalised and checked for uniqueness, and the status defaults to `DRAFT` when none is supplied.



**FIGURE 5.3 :** Sequence diagram of the “Create a project” use case

The authorship of a project is recorded but is deliberately not an association. The `createdBy` field stores the caller’s e-mail as text, stamped by the auditing layer, rather than a foreign key to `User`. Modelling it as a link would suggest a navigable relation the schema does not have, which is why no “created by” association appears in the class diagrams.

#### 5.2.2.4 Design decisions

**The lifecycle rule belongs to the domain, not to the controller.** The legal successors of a state are carried by the enumeration itself, through a `canTransitionTo` operation that each status answers for itself. The service consults it before any status change and rejects an illegal transition with HTTP 422. Had the rule been placed in the controller, a future entry point, an import or a scheduled job could have bypassed it silently.

**Archiving is a flag, not a state.** An archived project leaves the active lists while remaining restorable. Modelling it as a sixth state would have made it terminal, and a terminal state cannot be undone.

**The effective budget has a single writer.** A project carries an initial budget and, once amendments exist, a revised budget. Every computation reads the effective budget : the revised one when defined, the initial one otherwise. Only the amendment writes it. In the original workbook the budget was editable in several places, and the copies drifted ; a single writer makes that failure structurally impossible.

### 5.2.3 Sprint 3 realization

#### 5.2.3.1 Project identity and manager assignment

A project has exactly one active project manager at any time but keeps the history of previous assignments. Assigning a new manager closes the current assignment rather than overwriting it, so past workload remains attributable to the person who was responsible at the time. A partial unique index guarantees that only one assignment is active.

## 5.3 Sprint 4 — Team assignment and workload

### 5.3.1 Sprint 4 use case diagram

The sprint covers the use cases that feed the platform with the effort data : building the team, planning the monthly workload, submitting the effort actually consumed, and validating it. The developer appears here as an actor for the first time, since submitting one's own workload is the only write operation that role performs.

### 5.3.2 Sprint 4 analysis and design

#### 5.3.2.1 Sequence of the “Assign a developer” use case

Assignment is protected by two distinct checks : the caller must hold the `ASSIGN_DEVELOPER` permission *and* the target project must fall within their scope. The permission answers “what may I do” ; the scope answers “on which projects”.

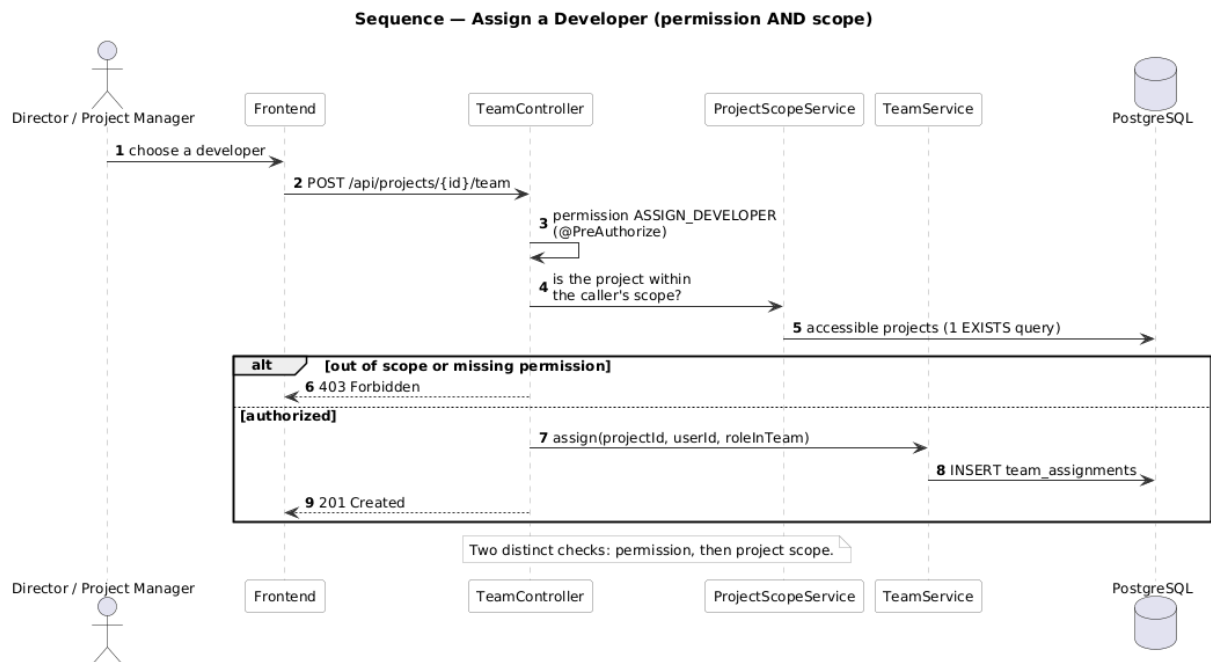


FIGURE 5.4 : Sequence diagram of the “Assign a developer” use case

#### 5.3.2.2 Design decisions

**Permission and scope are separate concerns.** Conflating them would let a project manager holding the permission act on a project that is not theirs. Keeping them distinct is what allows the same role to be granted broadly while its reach stays bounded by assignment.

**A monthly time series, not a grid of columns.** The workbook stores months as columns, which is why adding a year means editing the sheet structure. The platform stores one row per project, resource and month. The same information becomes queryable, aggregatable and extensible without schema change.

**A month is normalised to its first day.** Without normalisation, two entries for the same month recorded on different days would coexist silently and both would feed the indicators. Storing the first day of the month is what makes a unique index enforceable, and it is that uniqueness the whole financial engine relies on.

**A validated period is immutable.** Once the project manager validates a month, it can no longer be rewritten. A later correction cannot silently change a month on which a monthly review has

already been based. This is the direct answer to the traceability weakness of the Excel process.

### **5.3.3 Sprint 4 realization**

#### **5.3.3.1 Team assignment**

A team assignment links a user to a project with a role in the team and a validity period. A developer may belong to several projects simultaneously. Removing a member closes the assignment rather than deleting it.

#### **5.3.3.2 Planned and actual workload**

The model distinguishes the workload plan, which is forecast, from the actual workload, which was consumed. A developer submits their own workload on assigned projects only ; the project manager then validates it.

## **5.4 Sprint 5 — Agile planning**

### **5.4.1 Sprint 5 use case diagram**

The sprint covers the planning of the work itself : creating an iteration, maintaining a prioritised product backlog, committing items to an iteration, and following their progress on a board. Maintaining the plan is reserved to the project manager ; the director and the developer read it.

### **5.4.2 Sprint 5 analysis and design**

#### **5.4.2.1 Model of the backlog and its iterations**

A sprint belongs to a project and carries a goal, a start and end date, and a lifecycle (PLANNED, ACTIVE, CLOSED). A backlog item belongs to a project and *optionally* to a sprint, and carries a title, a priority, an estimate and a status.

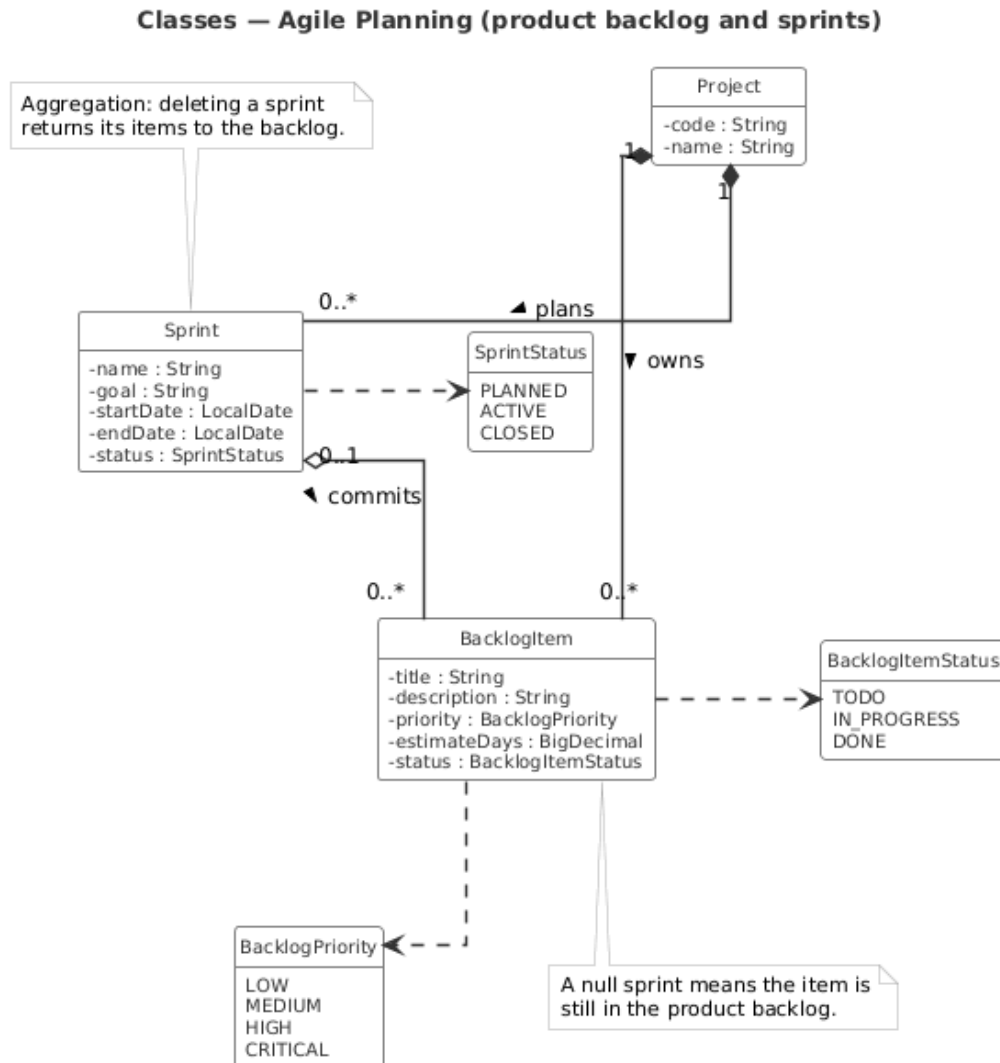


FIGURE 5.5 : Class diagram — agile planning

#### 5.4.2.2 Design decisions

**The link to a sprint is optional, and that is the model.** An item whose sprint is undefined is precisely an item in the product backlog : identified and estimated, but not yet committed. Making the association mandatory would have required a fictitious “unassigned” sprint per project, which carries no meaning and would have to be excluded from every query.

**Estimates are expressed in man-days, not in story points.** Story points are the more usual agile unit, but the whole platform already reasons in man-days : the workload plan, the loaded cost rate and the indicator engine all use them. An estimate in points would have been the only figure in the system that could not be compared with anything else. Man-days keep the backlog commensurable with the effort actually recorded.

**Deleting a sprint returns its items to the product backlog.** An iteration that is abandoned does not destroy the work identified for it ; the items simply become unplanned again, which is

the behaviour Scrum prescribes when a sprint is cancelled. The alternative, refusing deletion while items remain attached, was implemented first and rejected : besides being awkward, it left the items referencing a logically deleted sprint through a foreign key that was still valid.

**An item may never join the iteration of another project.** Assigning a sprint verifies that the sprint belongs to the same project as the item, and answers 404 otherwise. Without that check, a caller could attach an item to a foreign iteration simply by guessing an identifier, and the relation would cross the project boundary the whole authorization model is built on.

**The administrator receives neither permission.** `MANAGE_AGILE` is granted to the project manager and `VIEW_AGILE` to the director, the project manager and the developer. The administrator role governs the platform, not the content of projects, and the audit that rebuilt the authorization matrix deliberately reduced it to that. Granting it here would have quietly reintroduced what the audit removed.

### 5.4.3 Sprint 5 realization

#### 5.4.3.1 Iterations and the product backlog

The module exposes two nested resources under a project, one for iterations and one for backlog items, following the convention already used by governance and missions. Authorization is carried by the service layer rather than the controller, as elsewhere in the platform, so that no entry point can reach an unprotected method.

#### 5.4.3.2 The board

The board presents the items of the selected iteration in three columns derived from their status, alongside a column holding the product backlog. Moving a card calls a single endpoint that reassigns the iteration and the column together, because on a board those two actions are one gesture : dragging a card from the backlog into the current sprint changes both at once.

## Conclusion

Release 2 turned a secured shell into an operational tool : projects with an enforced lifecycle, teams with traceable assignments, workload captured as a reliable monthly time series, and the remaining work organised into a prioritised backlog and iterations. The data required by the financial engine is now available and trustworthy, which is the precondition for the indicators built in the next release.

# FINANCIAL MANAGEMENT AND GOVERNANCE

---

## Plan

|   |                                                          |    |
|---|----------------------------------------------------------|----|
| 1 | Release 3 backlog . . . . .                              | 54 |
| 2 | Sprint 6 — Billing and missions . . . . .                | 55 |
| 3 | Sprint 7 — Project governance . . . . .                  | 57 |
| 4 | Sprint 8 — Internal quote and indicator engine . . . . . | 58 |

## Introduction

This release delivers the value for which the platform was commissioned : replacing the financial logic of the Excel workbook. Sprint 6 records the revenue side, that is billing milestones, payments and amendments, together with the mission expenses that weigh on the margin. Sprint 7 adds the governance registers that document the life of a project. Sprint 8 closes the loop with the internal quote and the indicator engine, which consumes everything built since Release 1.

### 6.1 Release 3 backlog

The third release realises the user stories that carry the financial and governance value of the platform. It is the largest release of the project, and the one whose acceptance criterion is the most demanding : the figures it produces must match those of the reference workbook.

| Story | Need                                                                | Technical work                                                                             | Sprint |
|-------|---------------------------------------------------------------------|--------------------------------------------------------------------------------------------|--------|
| US-15 | Define billing milestones following contractual progress            | Milestone as a percentage of the effective budget, sum capped at 100% in the service layer | 6      |
| US-16 | Record payments received against a milestone                        | Payments kept as individual rows so partial settlements stay visible                       | 6      |
| US-17 | Record amendments so the revised budget reflects contractual change | Amendment as the single writer of the revised budget                                       | 6      |
| US-18 | Include travel costs in the project cost                            | Mission with typed components, each converted at the mission date before summation         | 6      |
| US-19 | Keep a risk register                                                | Risk with probability, impact, status and mitigation plan                                  | 7      |
| US-20 | Track deliverables and measure delivery progress                    | Deliverable with a controlled status, feeding the delivery rate of the indicator engine    | 7      |
| US-21 | Document stakeholders and change requests                           | Two registers composed by the project, edited by the project manager                       | 7      |

| Story | Need                                           | Technical work                                                                        | Sprint |
|-------|------------------------------------------------|---------------------------------------------------------------------------------------|--------|
| US-22 | Give the project an economic baseline          | Internal quote structured by section, gated by a dedicated permission                 | 8      |
| US-23 | Compute the financial indicators automatically | Recompute-on-read engine valuing effort at the cost rate of its own year              | 8      |
| US-24 | Freeze a monthly review as history             | Immutable snapshot storing the indicators and the earned value entered by the manager | 8      |
| US-25 | Supervise the whole portfolio                  | Per-project and portfolio dashboards derived from the same engine                     | 8      |

**TABLEAU 6.1** : Backlog of release 3

## 6.2 Sprint 6 — Billing and missions

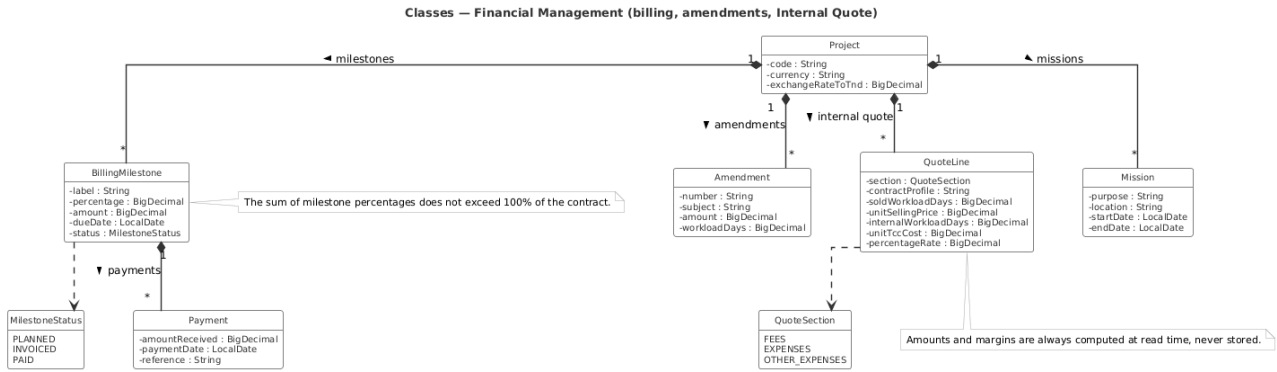
### 6.2.1 Sprint 6 use case diagram

The sprint covers the use cases through which a project earns and spends outside labour : defining a billing milestone, invoicing it, recording a payment against it, recording a contractual amendment, and recording a mission with its expenses. All are reserved to the project manager, except the amendment, which the director records because it changes the contract.

### 6.2.2 Sprint 6 analysis and design

#### 6.2.2.1 Model of billing and missions

A milestone belongs to one project and carries a label, a percentage of the contract, an amount, a due date and a status. A mission belongs to a project and an employee, and decomposes into typed cost components.



### 6.2.2.2 Design decisions

**The milestone amount is derived, not stored.** It follows from the effective budget and the percentage, so an amendment propagates automatically to every milestone not yet invoiced. Storing the amount would have required updating each milestone on every amendment, and any missed update would have gone unnoticed.

**The sum of percentages may not exceed 100%.** This rule is inherited from the workbook, where it was a convention the spreadsheet could not enforce. Here it is checked in the service layer, and an attempt to exceed it is rejected rather than silently accepted.

**Payments are rows, not a settled total.** Keeping a single settled amount would lose the history of partial settlements, which is precisely what a project manager chasing collection needs to see.

**Conversion happens before summation, never after.** A mission may be incurred in a currency different from the reporting one. Each component is converted at the rate of the mission date and only then summed. Summing first would add amounts expressed in different currencies, and the total would be meaningless while still looking plausible.

## 6.2.3 Sprint 6 realization

### 6.2.3.1 Milestone lifecycle

A milestone moves through three states : planned, invoiced and paid. Invoicing records the invoice date ; a payment recorded against an invoiced milestone moves it to paid.

### 6.2.3.2 Amendments

An amendment carries a reference, an amount and an impact on the sold workload. It is the single writer of the revised budget introduced in Sprint 3, which is what keeps the contractual and financial history coherent.

## 6.3 Sprint 7 — Project governance

### 6.3.1 Sprint 7 use case diagram

The sprint covers the four registers that document the qualitative life of a project : risks, deliverables, stakeholders and change requests. The project manager maintains them ; the director consults them.

### 6.3.2 Sprint 7 analysis and design

#### 6.3.2.1 Model of the governance registers

The four registers share the same shape : each is composed by the project and governed by a small business enumeration.

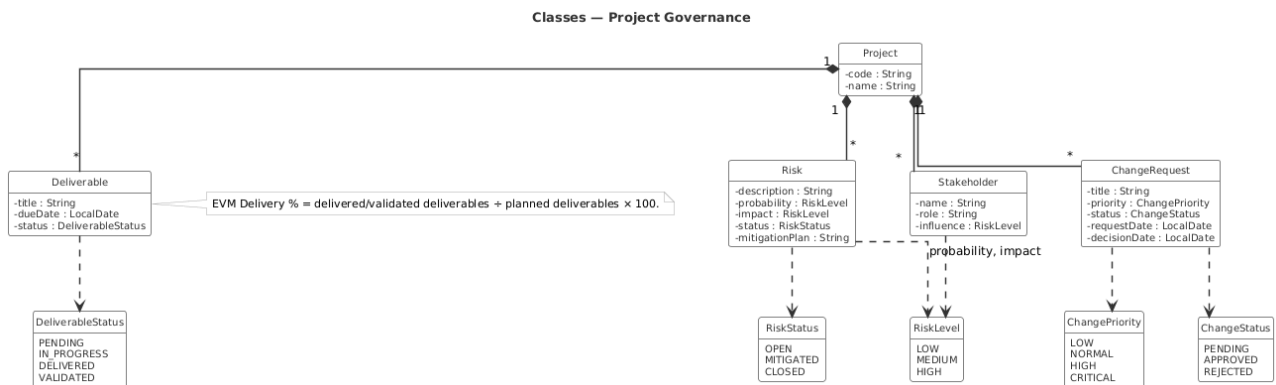


FIGURE 6.2 : Class diagram — project governance

#### 6.3.2.2 Design decisions

**The registers are attached to the project, not kept beside it.** Their value lies less in their individual complexity than in belonging to the project itself, which the spreadsheet process could not guarantee : risks and deliverables lived in separate documents that drifted from the workbook.

**A deliverable carries a controlled status, not free text.** This is not documentation tidiness. The delivery rate consumed by the indicator engine of Sprint 8 is derived from this register, as the ratio of delivered and validated deliverables to planned ones. A free-text status could not be counted, and the indicator would have had no input.

**Deletion is logical.** Removing an entry deactivates it and preserves the history, so that a risk closed months ago remains visible in the record of how the project was steered.

### 6.3.3 Sprint 7 realization

Each register exposes the same nested-resource shape under a project, with authorization carried by the service layer and a dedicated pair of permissions separating consultation from modification.

## 6.4 Sprint 8 — Internal quote and indicator engine

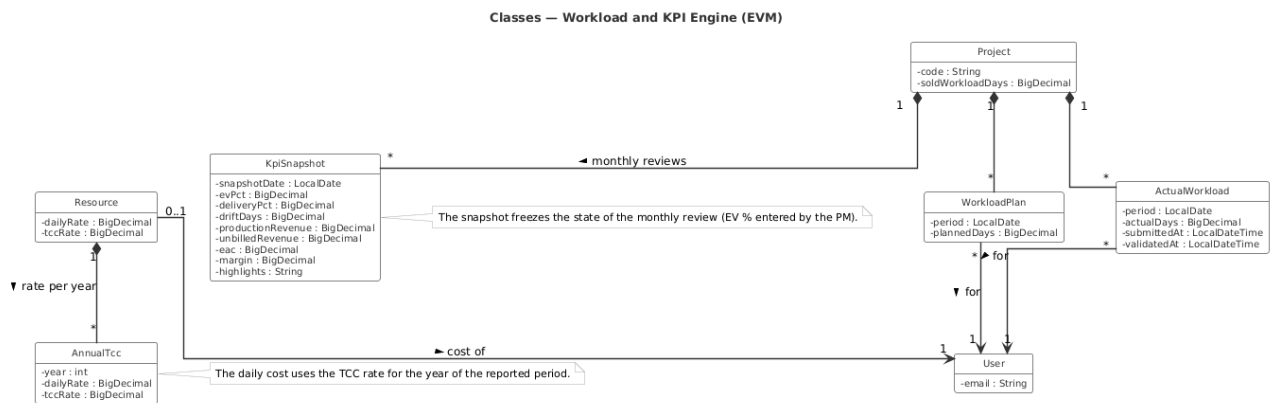
### 6.4.1 Sprint 8 use case diagram

The sprint covers the use cases that produce the figures the platform exists for : defining the internal quote, consulting the computed indicators, freezing a monthly review, and consulting the portfolio dashboard. The internal quote is reserved to the director alone.

### 6.4.2 Sprint 8 analysis and design

#### 6.4.2.1 Model of the indicator engine

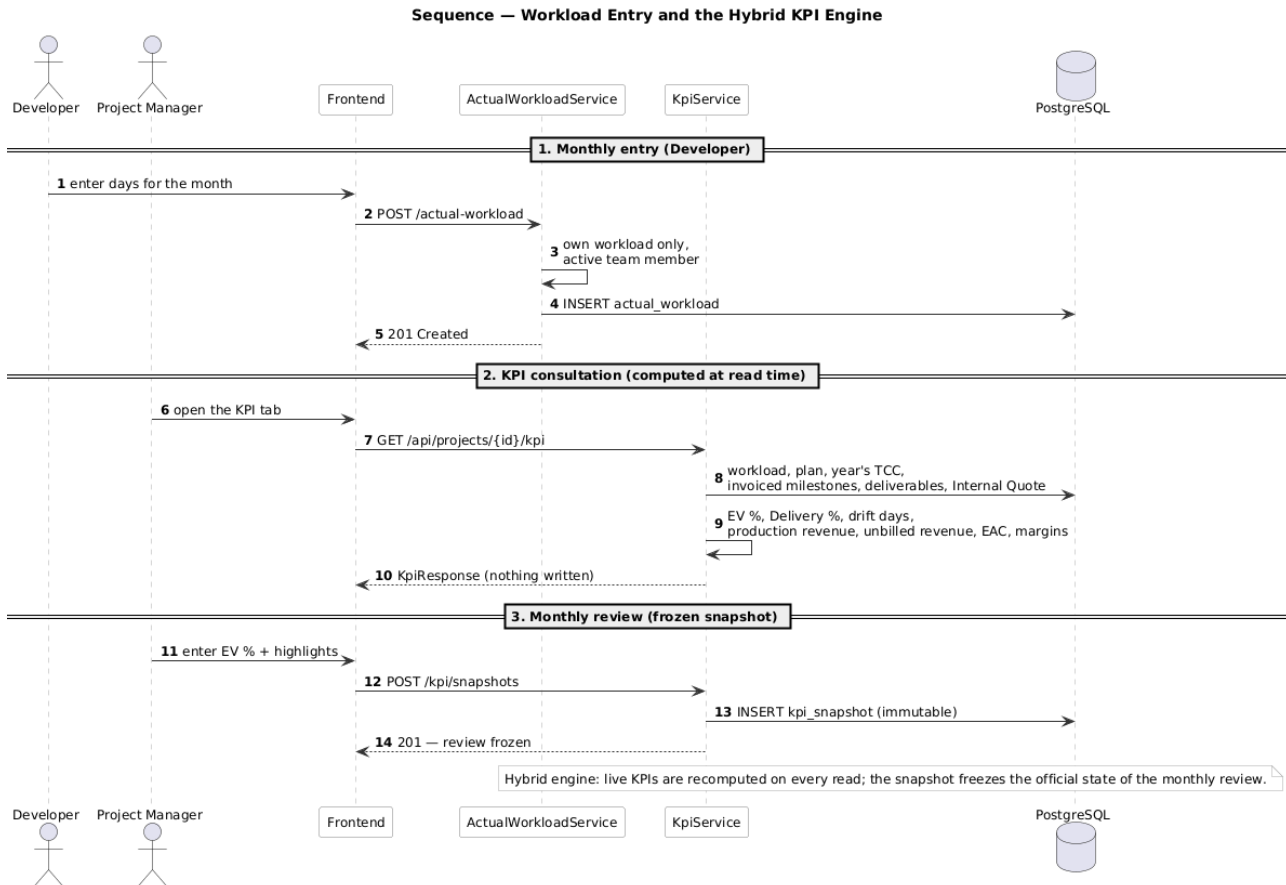
The engine reads from every module built since Release 1 : workload for consumed and remaining effort, cost rates for valuation, billing for invoiced revenue, and the deliverable register for delivery progress.



**FIGURE 6.3 :** Class diagram — workload and indicator engine

#### 6.4.2.2 Sequence from workload entry to frozen review

Figure 6.4 shows the complete chain, from the submission of a month of effort to the snapshot that freezes the monthly review.



**FIGURE 6.4 :** Sequence diagram — workload entry and indicator computation

### 6.4.2.3 Design decisions

**Amounts and margins are never stored.** They are recomputed at read time from quantities, unit values and the exchange rate. Persisting them would duplicate commercially sensitive figures and allow the stored value to diverge from its inputs, which is exactly the failure the workbook suffered when a formula was overwritten by a pasted constant.

**A read has no side effect.** Consulting a dashboard writes nothing. This is what makes the indicators safe to display anywhere, and it is why two consecutive reads always agree.

**The earned value is entered, not computed.** The percentage of value earned is a judgement made by the project manager during the monthly review, not a measurement the system can derive. Pretending otherwise would have produced an indicator that looks objective and is not.

**The monthly review is frozen and immutable.** Submitting the review creates a snapshot of the indicators of that month. This is the digital equivalent of the monthly column of the statistics sheet : live figures for steering, frozen figures for history. A later correction cannot rewrite a review a decision was already based on.

**The internal quote is gated by its own permission.** Unit costs and internal margins are the most commercially sensitive data the platform holds, so access is not inherited from the project

role but granted explicitly and separately.

### 6.4.3 Sprint 8 realization

#### 6.4.3.1 The internal quote

The quote is the economic baseline of the project : line items grouped by section, each carrying a sold quantity and unit price on the revenue side, and an internal quantity and unit cost on the cost side. The computation runs in two passes so that section subtotals and the overall margin remain consistent with the line items they summarise.

#### 6.4.3.2 The computed indicators

On each read the engine recomputes, from live data, the indicators of the reference workbook :

- **Actual cost** — consumed man-days valued at the cost rate of their own year ;
- **ETC** — remaining man-days valued the same way ;
- **EAC** — actual cost plus ETC ;
- **Production revenue** — contract total weighted by the earned value percentage ;
- **Accrued income** — production revenue minus the invoiced total ;
- **Margin** — effective budget minus EAC, in value and in percentage ;
- **Drift** — sold workload minus consumed minus remaining.

## Conclusion

Release 3 delivered the core value of PMS : billing and expenses, the governance registers, the internal quote and above all the indicator engine that replaces the two thousand formulas of the reference workbook. The hybrid design, live computation for steering and frozen snapshots for history, reproduces the strengths of the spreadsheet while removing its fragility.

# CONSOLIDATION, DELIVERY AND DEPLOYMENT

---

## Plan

|   |                                                                    |    |
|---|--------------------------------------------------------------------|----|
| 1 | Release 4 backlog . . . . .                                        | 62 |
| 2 | Sprint 9 — Interface consolidation, testing and security . . . . . | 63 |
| 3 | Sprint 10 — Containerization and continuous delivery . . . . .     | 65 |
| 4 | Final state of the product . . . . .                               | 67 |

## Introduction

The functional scope of the platform was completed at the end of Release 3. This final release makes it deliverable : a coherent and accessible user interface, a verified authorization policy, a security review, and a reproducible deployment. It gathers two sprints : Sprint 9 consolidates the interface and verifies the platform, Sprint 10 makes it reproducibly deployable and continuously verified. Neither delivers new business functionality ; both deliver deliverability.

### 7.1 Release 4 backlog

The last release carries a single user story. The remainder of its content is cross-cutting quality work that no functional story expresses, which is itself characteristic of a consolidation release.

| Story | Need                                                          | Technical work                                                                                             | Sprint |
|-------|---------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|--------|
| US-26 | A consistent, accessible and responsive interface             | Design system of tokens and shared components applied across every module                                  | 9      |
| —     | Verify that the authorization model behaves as designed       | Automated suite asserting, for each protected endpoint, that the permission grants and its absence refuses | 9      |
| —     | Review the application against the common web vulnerabilities | Category-by-category review with the measure taken recorded for each                                       | 9      |
| —     | Deploy reproducibly on any host                               | Container image per component, orchestration with health probes and startup ordering                       | 10     |
| —     | Verify every change automatically                             | Continuous integration pipeline building, testing and assembling the stack                                 | 10     |

**TABLEAU 7.1** : Backlog of release 4

## 7.2 Sprint 9 — Interface consolidation, testing and security

### 7.2.1 Sprint 9 use case diagram

This sprint adds no use case. It acts on those already delivered, by unifying the screens through which they are reached and by verifying that the authorization attached to each of them behaves as specified. The relevant view is therefore not a new use case diagram but the global one of Chapter 2, now entirely covered by the interface.

### 7.2.2 Sprint 9 analysis and design

#### 7.2.2.1 Where authorization is decided

The interface consumes the permission list returned by the context endpoint and derives from it both the navigation menu and the visibility of individual actions. A button the user is not entitled to use is not merely disabled : it is absent.

This must be understood correctly, and the report states it explicitly : hiding a control is an **ergonomic** measure, not a security measure. The authorization decision is taken by the server on every request, as described in Sprint 2. The client-side rule exists so that the user is never offered an action that would be refused ; it is never the thing that refuses it.

#### 7.2.2.2 Design decisions

**A design system rather than eight sets of screens.** The modules were built successively, each with its own screens. Rather than harmonising them by hand, a token-based visual language, covering colours, typography, spacing and elevation, is declared once and consumed everywhere, with a set of shared components reused rather than reimplemented. Harmonising by hand would have left the next module free to diverge again.

**Testing concentrates on what a regression makes expensive.** Not everything is worth an automated test. The suite covers the business rules and the authorization model : lifecycle transitions, the billing ceiling, workload uniqueness and immutability, multi-currency conversion, and the indicator formulas compared against the reference workbook.

**Access control is tested in both directions.** For each protected endpoint the suite asserts that a caller holding the permission succeeds *and* that a caller without it is refused. Testing only the success path would leave the platform's central claim unverified : it is the refusal that proves the policy is enforced.

### 7.2.3 Sprint 9 realization

#### 7.2.3.1 Frontend structure and session handling

The client is a single-page application organised by feature, mirroring the backend modules. The access token obtained at sign-in is kept in memory and attached to every request by an HTTP interceptor ; the refresh token remains in its `HttpOnly` cookie, unreachable from JavaScript. When a request returns 401, the interceptor attempts a single refresh before returning the user to the sign-in screen.

#### 7.2.3.2 Interface consolidation

Beyond the visual language, the consolidation addressed accessibility and consistency : sortable table headers announcing their state, labelled icon-only controls, explicit empty states in place of blank tables, and a responsive layout. The interface being the only part of the platform its users ever see, this convergence matters as much to adoption as the correctness of the engine behind it.

#### 7.2.3.3 Security review

The application was reviewed against the common categories of web vulnerability.

| Category                            | Measure taken                                                                                                                                                                   |
|-------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Broken access control               | Every endpoint annotated with its required permission ; scope checked independently of permission ; no role name tested in code                                                 |
| Cryptographic failures              | Passwords hashed with BCrypt ; tokens signed ; refresh token confined to an <code>HttpOnly</code> , <code>Secure</code> , <code>SameSite</code> cookie and rotated on every use |
| Injection                           | Parameterised queries through the ORM ; no string-built SQL                                                                                                                     |
| Identification and session failures | Short-lived access token ; instant revocation by token version ; forced password change on first login ; repeated failures throttled                                            |
| Sensitive data exposure             | Financial data filtered by permission ; internal quote behind a dedicated permission ; computed amounts never persisted                                                         |

| Category                  | Measure taken                                                                                                          |
|---------------------------|------------------------------------------------------------------------------------------------------------------------|
| Security misconfiguration | Schema managed by versioned migrations with the ORM in validation mode ; secrets externalised as environment variables |

**TABLEAU 7.2** : Security review by vulnerability category

#### 7.2.3.4 Packaging, and what it does not solve

At the close of this sprint the platform is packaged as an executable archive for the backend and a static build for the frontend, with the schema brought up to date by the migration tool at startup and all configuration supplied by environment variables. Startup is idempotent : deploying the same artefact against an up-to-date database changes nothing, and against an older one applies exactly the missing migrations.

This is sufficient to *run* the application but not to *deploy* it reproducibly : it still assumes a host on which the right Java version, the right Node version and a correctly provisioned PostgreSQL instance are already present. Sprint 10 removes that assumption.

### 7.3 Sprint 10 — Containerization and continuous delivery

#### 7.3.1 Sprint 10 use case diagram

Like the previous one, this sprint adds no business use case. Its actor is not a user of the platform but the person deploying it, and its scenarios are building the images, starting the stack and verifying a change automatically. The deployment view of Chapter 3 is the relevant diagram.

#### 7.3.2 Sprint 10 analysis and design

##### 7.3.2.1 A reconsidered architecture decision

The project had initially recorded a decision to run natively, without containers, justified by iteration speed on an already-provisioned machine. That decision served development well but left two gaps : no reproducible deployment, and a production profile that had never actually been executed.

The decision was therefore superseded rather than silently contradicted : a new record documents the move to containers, states what it changes, and confirms what it does *not* change. In particular, three containers are **not** three microservices : the modular monolith decision stands. What is containerized is one application, its database and its static server.

### 7.3.2.2 Design decisions

**Two build stages, so the toolchain never reaches the runtime image.** The backend image contains a Java runtime and the application archive, with neither Maven nor sources; the frontend image contains a web server and the compiled assets, with no Node runtime.

**Tests are not run during image construction.** Building an image is a packaging step; the quality gate belongs to the pipeline. Running the suite in both places would double build time without producing any additional signal.

**A single origin through the reverse proxy.** The web server serves the application and forwards `/api` to the backend container, so the browser contacts one origin. The cross-origin configuration required by the development setup becomes unnecessary, and the refresh cookie, restricted to its own path and marked `SameSite=Strict`, works without any exception. On this point the containerized deployment is simpler and safer than the development loop.

**Ordering by health probe, not by declaration order.** A container being *started* is not the same as a container being *ready*. The database is probed over TCP, because the temporary server used during initialisation accepts only local socket connections and would otherwise be reported ready too early. The backend exposes a health endpoint, deliberately unauthenticated but returning no detail, with a generous startup window because its first run applies the full set of schema migrations.

## 7.3.3 Sprint 10 realization

### 7.3.3.1 Defects revealed by the exercise

Preparing the pipeline exposed three pre-existing defects that native execution had concealed. The most consequential concerned the frontend build configuration : the production configuration never substituted the production environment file, so the production bundle embedded the development API address. On the development machine the application appeared to work, because the bundle was calling the local backend, while it would have been broken on any other host. The second was a build wrapper documented in the project files but absent from the repository. The third was an ignore rule that silently excluded the configuration template from version control, precisely the file the deployment procedure requires.

That these defects surfaced only when the delivery chain was automated is itself the argument for automating it.

### 7.3.3.2 Pipeline

The pipeline separates what must always hold from what is only meaningful on integration branches. Compilation, the backend test suite, the production frontend build and the construction of both images run on every proposed change : a component that only breaks after merging is not a gate. The end-to-end test, which starts the full stack and performs a real authentication through the reverse proxy, runs on integration branches, where its cost is justified. Image publication is reserved for the main branch.

The frontend job also carries an explicit guard against the defect described above : it fails if the development address reappears in the production bundle.

## 7.4 Final state of the product

At the end of the ten sprints, PMS covers the whole scope identified during the analysis of the Excel workbook : dynamic authorization, project and team management, workload planning and recording, agile planning, billing, missions, governance and the earned-value indicator engine, all behind a unified interface. Table 7.3 summarises the path from the reference workbook to the delivered platform.

| Sprint   | Increment delivered                                 | Excel logic replaced                              |
|----------|-----------------------------------------------------|---------------------------------------------------|
| Sprint 1 | Technical foundation, authentication                | — (no spreadsheet equivalent)                     |
| Sprint 2 | Dynamic authorization, users, resources, cost rates | TCC sheet                                         |
| Sprint 3 | Projects and lifecycle                              | Identification sheet                              |
| Sprint 4 | Teams, workload plan and actuals                    | Forecast cost and actual cost sheets              |
| Sprint 5 | Agile planning : product backlog and iterations     | — (no spreadsheet equivalent)                     |
| Sprint 6 | Billing, payments, amendments, missions             | Billing progress and missions sheets              |
| Sprint 7 | Governance registers                                | Risks, deliverables, stakeholders, changes sheets |
| Sprint 8 | Internal quote and indicator engine                 | Statistics and dashboard sheets                   |

| Sprint    | Increment delivered                             | Excel logic replaced |
|-----------|-------------------------------------------------|----------------------|
| Sprint 9  | Interface consolidation, tests, security review | — (quality)          |
| Sprint 10 | Containerization and continuous delivery        | — (delivery chain)   |

**TABLEAU 7.3 :** From the reference workbook to the delivered platform

## Conclusion

Release 4 closed the construction of the platform by addressing what functionality alone does not deliver : a coherent interface, a verified security model and a reproducible deployment. The incremental path followed since Release 1 (foundation, operations, finance, consolidation) produced a platform whose every increment was demonstrable in isolation, and whose final state covers the entire business logic previously carried by the Excel workbook.

# General Conclusion and Perspectives

The objective of this final year project was to replace a scattered, spreadsheet-based project steering process with a centralised platform that preserves the richness of the existing financial model while removing its fragility. That objective is met : PMS covers the whole functional scope identified during the analysis of the reference workbook, from dynamic authorization to the earned-value indicator engine.

## Assessment

The construction proceeded through ten increments, each demonstrable in isolation. Release 1 delivered the technical foundation, authentication and above all the dynamic authorization model. Release 2 opened operational steering with projects, teams and workload. Release 3 delivered the core business value : billing, missions, governance and the indicator engine. Release 4 consolidated the interface, verified the security model, and made the platform reproducibly deployable through containerization and a continuous integration pipeline.

Three design decisions structure the result and are, in retrospect, the ones that mattered most.

The first is the **dynamic authorization model**. By inserting a permission level between roles and modules, and by keeping the role–permission matrix in the database, the platform makes its security policy configurable data rather than compiled code. Granting a new right is an administrative act, not a release.

The second is the **hybrid indicator engine**. Rather than choosing between recomputing indicators on demand and persisting them, the platform does both : live computation guarantees that no dashboard displays a stale figure, while a frozen monthly snapshot preserves the official state of each review. This reproduces the strength of the original workbook, a live dashboard with monthly history, without its fragility.

The third is the **normalisation of the monthly time series**. Replacing a grid of month columns with one row per project, resource and month turned a structural limitation of the spreadsheet into a simple insertion, and made integrity constraints expressible : no duplicate month, no entry on an unassigned project, no silent rewriting of a validated period.

## Skills Acquired

Beyond the technologies used, this project required reading a real business model out of an artefact that was never written as a specification, and arbitrating between what the workbook did,

what the written specifications said, and what the code could reasonably guarantee. It also required designing for confidentiality : deciding that commercially sensitive amounts should be computed on read rather than stored is an engineering decision with a business justification, not a technical preference.

## Limitations and Perspectives

Several avenues remain open.

**Automatic time-tracking import.** The workload model was designed to accept a second data source alongside manual entry, so that effort recorded in an external time-tracking tool could be imported and reconciled. The mechanism is anticipated in the design but not implemented ; it is the most immediate extension.

**Reporting and export.** The platform computes and displays the indicators ; generating exportable review documents in the format the company already circulates would shorten the last manual step of the monthly review.

**Integration with the information system.** The business rules explicitly anticipate future integration points with accounting, human resources and enterprise systems. The modular organisation of the code was chosen with that reversibility in mind : a module can be extracted behind an interface without restructuring the whole.

**Scaling the deployment.** The current deployment targets the internal use of one company. A significant growth in the number of projects or users would justify revisiting the modular monolith, a decision deliberately taken as reversible rather than permanent.

The platform delivered is therefore both an answer to an immediate operational need and a foundation on which the company's project steering can continue to evolve.

# Webographie

- [1] « Spring Boot Reference Documentation. » [Accessed August 2026]. adresse : <https://docs.spring.io/spring-boot/documentation.html>
- [2] « Spring Security Reference — Authentication and Authorization. » [Accessed August 2026]. adresse : <https://docs.spring.io/spring-security/reference/>
- [3] « Spring Data JPA Reference Documentation. » [Accessed August 2026]. adresse : <https://docs.spring.io/spring-data/jpa/reference/>
- [4] « Spring Boot Actuator — Production-ready Features. » [Accessed August 2026]. adresse : <https://docs.spring.io/spring-boot/reference/actuator/index.html>
- [5] « MapStruct Reference Guide — Java Bean Mappings. » [Accessed August 2026]. adresse : <https://mapstruct.org/documentation/stable/reference/html/>
- [6] « PostgreSQL 17 Documentation. » [Accessed August 2026]. adresse : <https://www.postgresql.org/docs/17/index.html>
- [7] « Flyway Documentation — Database Migrations as Code. » [Accessed August 2026]. adresse : <https://documentation.red-gate.com/flyway>
- [8] « Angular Documentation. » [Accessed August 2026]. adresse : <https://angular.dev/overview>
- [9] « Angular — Signals and Reactive State. » [Accessed August 2026]. adresse : <https://angular.dev/guide/signals>
- [10] « Transloco — Internationalization Library for Angular. » [Accessed August 2026]. adresse : <https://jsverse.gitbook.io/transloco>
- [11] « Bootstrap 5 Documentation. » [Accessed August 2026]. adresse : <https://getbootstrap.com/docs/5.3/getting-started/introduction/>
- [12] « Chart.js Documentation. » [Accessed August 2026]. adresse : <https://www.chartjs.org/docs/latest/>
- [13] « RFC 7519 — JSON Web Token (JWT). » [Accessed August 2026]. adresse : <https://www.rfc-editor.org/rfc/rfc7519>
- [14] « OWASP Top 10 — Web Application Security Risks. » [Accessed August 2026]. adresse : <https://owasp.org/www-project-top-ten/>
- [15] « OWASP Cheat Sheet Series — Authentication and Session Management. » [Accessed August 2026]. adresse : <https://cheatsheetseries.owasp.org/>

- [16] « Docker — Dockerfile Reference and Multi-stage Builds. » [Accessed August 2026]. adresse : <https://docs.docker.com/reference/dockerfile/>
- [17] « Docker Compose — Compose File Specification. » [Accessed August 2026]. adresse : <https://docs.docker.com/reference/compose-file/>
- [18] « nginx Documentation — HTTP Proxy Module. » [Accessed August 2026]. adresse : [https://nginx.org/en/docs/http/nginx\\\_http\\\_proxy\\\_module.html](https://nginx.org/en/docs/http/nginx\_http\_proxy\_module.html)
- [19] « GitHub Actions Documentation — Workflow Syntax. » [Accessed August 2026]. adresse : <https://docs.github.com/en/actions>
- [20] « Apache Maven — Introduction to the Build Lifecycle. » [Accessed August 2026]. adresse : <https://maven.apache.org/guides/introduction/introduction-to-the-lifecycle.html>
- [21] « OMG Unified Modeling Language (UML) Specification, version 2.5.1. » [Accessed August 2026].  
adresse : <https://www.omg.org/spec/UML/2.5.1/>
- [22] « PlantUML Language Reference Guide. » [Accessed August 2026]. adresse : <https://plantuml.com/guide>
- [23] « The Scrum Guide — The Definitive Guide to Scrum. » [Accessed August 2026]. adresse : <https://scrumguides.org/scrum-guide.html>
- [24] « Project Management Institute — Earned Value Management. » [Accessed August 2026]. adresse : <https://www.pmi.org/learning/library/earned-value-management-systems-analysis-8026>

# Résumé

FR

**Mots clés :** KW1, KW2

# Abstract

EN

**Keywords :** KW1, KW2